INTERVIEWHUB · INTERVIEW PREP

CI/CD Pipelines — InterviewHub

Platform, SRE & DevOps interview guide

Questions	**86 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/cicd_interview_guide.html**
Printed	**2026-10-23**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 What is CI/CD — difference between continuous integration, delivery, and deployment?

Fundamentals

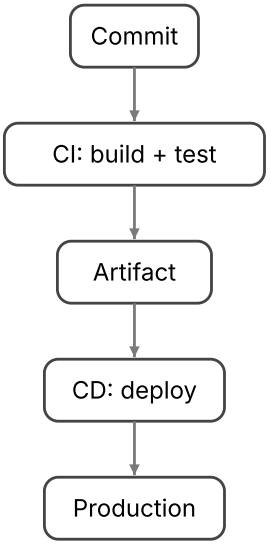
🎙 Spoken answer · ~80 s

All three are stages of automating software delivery. Continuous integration means developers merge changes into a shared branch frequently, and every merge triggers an automated build and tests, so integration problems surface within minutes. Continuous delivery extends that: every change that passes the pipeline produces a release-ready artefact that could be deployed to production at any time, but the final push to production is a manual decision, perhaps with an approval. Continuous deployment goes one step further: every change that passes all automated checks is deployed to production automatically, with no human gate. The difference between delivery and deployment is the manual step. The choice depends on risk tolerance, regulation, and the strength of automated testing and monitoring. Many teams practise continuous delivery for sensitive systems and continuous deployment for lower-risk services. What makes either possible is a fast reliable pipeline, strong automated tests, feature flags to separate deploy from release, progressive rollouts with automated rollback, and good observability. The benefits are smaller batches, faster feedback, lower risk per change, and quicker recovery. I measure it with the DORA metrics: deployment frequency, lead time, change failure rate, and time to restore.

Likely follow-ups

- What is the practical difference between delivery and deployment?
- Which metrics show your CI/CD is healthy?

🗺 CI vs CD




```
CI: merge + build + test on every commit (catch integration bugs early)
```

```
CD (Delivery): artifact is always releasable; deploy is a manual trigger
```

```
CD (Deployment): every passing commit auto-deploys to production
```

certs.eknathalabs.com

CI catches bugs when they are cheap to fix. Continuous Delivery ensures the pipeline always produces a deployable artifact. Continuous Deployment removes all manual gates. Most teams use CI + Delivery with a manual production gate as the sweet spot.

DORA metrics: Deployment frequency, lead time for changes, change failure rate, MTTR. Elite teams deploy multiple times/day with under 1 hour lead time. Your CI/CD design directly determines these numbers.

📚 **Official docs**

- Martin Fowler: Continuous Delivery

Q2 What is an immutable artifact — why do you never rebuild for production? Fundamentals

🎙 **Spoken answer · ~81 s**

An immutable artefact is a build output, such as a container image or package, that is built once, given a unique version, and never modified afterwards. The same exact artefact is then promoted through every environment: dev, staging, and production. You never rebuild for production because a rebuild can produce a different result: dependencies may have changed, the base image may have been updated, the build environment may differ, or flags may differ. Then what you tested in staging isn't what you deploy, and the tests lose their meaning. Rebuilding also makes it hard to reproduce and trace issues. The practice: build in CI, tag the artefact with the commit SHA or a semantic version, not a mutable tag like latest, push it to a registry, ideally enabling tag immutability, and record its digest. Environment-specific differences are injected at deploy time through configuration, environment variables, or secrets, never baked in. Benefits: reproducibility, confidence that tested equals deployed, fast and reliable rollback by redeploying a previous artefact, auditability, and the ability to sign and verify provenance. Deploying by digest is the strictest form. Together, build once and promote everywhere is a core principle of reliable delivery.

Likely follow-ups

- Why is the latest tag a problem?
- How do you handle environment differences without rebuilding?

```
docker build -t myapp:${GITHUB_SHA} .   # tag = git commit SHA
```

```
docker push registry/myapp:${GITHUB_SHA}
```

```
# Same image promoted through all stages:
```

```
registry/myapp:abc1234 → staging → production (never rebuilt)
```

```
docker tag registry/myapp:$SHA registry/myapp:v1.2.3  # tag on release
```

certs.eknathalabs.com

Build once, deploy everywhere. Never rebuild for production — different environment, compiler version, or timestamp can produce a different binary. The image that passed staging tests is exactly the binary deployed to production. Use Git SHA as the image tag for traceability: any deployed image maps directly to its source commit.

📚 **Official docs**

- Martin Fowler: Deployment Pipeline

Q3 How do you structure secrets in CI/CD pipelines — what are the best practices?

Fundamentals

🎙 **Spoken answer · ~84 s**

Secrets in pipelines need to be protected at rest, in use, and in output. Best practices: never commit secrets to the repository, and scan for them with pre-commit hooks and CI scanners. Store them in the CI platform's encrypted secret store, scoped to the narrowest level, such as per environment, with production secrets available only to protected branches and approved jobs. Better still, avoid long-lived secrets: use OIDC federation so the pipeline obtains short-lived cloud credentials, with roles limited to what the job needs; or fetch secrets at runtime from a manager like Vault, AWS Secrets Manager, or Google Secret Manager. Apply least privilege: separate credentials for build, test, and deploy, and different ones per environment. Mask secrets in logs, avoid echoing them or passing them on the command line, and disable debug tracing that might print them. Don't expose secrets to pull requests from forks. Rotate regularly and immediately after any suspected leak, and audit access. Pin third-party actions or plugins to specific versions or commit hashes to prevent supply chain attacks that steal secrets. Inject secrets at deploy time rather than baking them into images. Monitor for anomalous use. A strong pipeline has almost no static secrets at all.

Likely follow-ups

- How does OIDC federation avoid stored cloud keys?
- Why restrict production secrets to protected branches?

certs.eknathalabs.com

```
# GitHub Actions: repository secrets (encrypted)
```

```
env: {API_KEY: "${{ secrets.API_KEY }}"}
```

```
# GCP Workload Identity Federation (no keys at all):
```

```
- uses: google-github-actions/auth@v2
```

```
  with:
```

```
    workload_identity_provider:
projects/123/locations/global/workloadIdentityPools/pool/providers/github
```

```
    service_account: deploy@project.iam.gserviceaccount.com
```

Hierarchy: hardcoded (never) → env var in CI settings (basic) → secrets manager (good) → Workload Identity/OIDC (best — no static credentials). Workload Identity Federation: GitHub OIDC token exchanges for a short-lived GCP access token. No service account key files anywhere. This is the production standard for GitHub Actions + GCP pipelines.

📚 **Official docs**

- GitHub Actions: Security hardening

Q4 How does GitHub Actions work — workflows, jobs, and steps?

GitHub Actions

🎙 **Spoken answer · ~81 s**

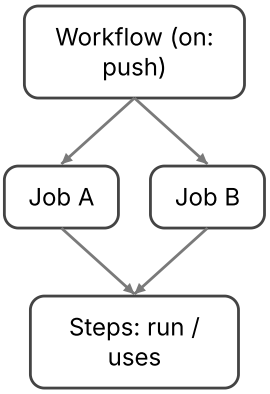
GitHub Actions is GitHub's built-in automation platform. A workflow is a YAML file in the .github/workflows directory, triggered by events such as push, pull request, schedule, manual dispatch, or another workflow. A workflow contains one or more jobs. Jobs run on runners, which are virtual machines hosted by GitHub or self-hosted, and by default run in parallel, though you can order them using needs. Each job consists of steps that run sequentially on the same runner and share its filesystem; a step is either a shell command or an action, a reusable unit from the marketplace or your own repository, such as checkout or setup-node. Data moves between jobs through artefacts, caches, and outputs, since each job has a fresh environment. Other features: matrix strategies to run a job across combinations of versions or platforms, environments with protection rules and required reviewers, secrets and variables at repository, environment, and organisation levels, concurrency groups to cancel or queue runs, conditionals, and reusable workflows. Permissions are controlled with the GITHUB_TOKEN, which should be given the least privilege. Good practice: pin actions by commit SHA, cache dependencies, keep jobs small and fast, and use OIDC for cloud access.

Likely follow-ups

- How do jobs share data given each runs on a fresh machine?
- What is a matrix strategy?

certs.eknathalabs.com

🗺 **GitHub Actions**




```
on:
  push: {branches: [main]}
  pull_request: {branches: [main]}
  push: {tags: ["v*.*.*"]}
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - run: make test
  deploy:
    needs: test
    environment: production  # requires manual approval
```

Workflows trigger on events. Jobs run in parallel by default — use needs: for ordering. environments add manual approval gates and environment-specific secrets. Matrix builds run the same job across multiple configs (Node 18/20/22) simultaneously. Actions are reusable steps (uses:).

📚 **Official docs**

- GitHub Actions documentation

certs.eknathalabs.com

Q5 How do you implement a full GitHub Actions pipeline for a containerised app?

GitHub Actions

🎙 Spoken answer · ~85 s

A complete pipeline for a containerised app has stages: build and test, scan, publish, and deploy. Trigger it on pull requests, for validation, and on pushes to main, for release. In the first job, check out the code, set up the language toolchain with dependency caching, run linting and unit tests, and publish results. A second job builds the image with Docker Buildx, using layer caching, tagged with the commit SHA. Scan it with Trivy or similar, and fail on critical vulnerabilities; generate an SBOM. On main, push the image to a registry, authenticating through OIDC so no long-lived credentials are stored, and optionally sign it with Cosign. Then deploy: a job targeting a staging environment applies the manifests or Helm chart, referencing the image digest, followed by smoke tests. A production job uses a GitHub environment with required reviewers, deploys, runs health checks, and rolls back automatically on failure. Use needs to order jobs, set minimal permissions, use concurrency groups to avoid overlapping deployments, and reusable workflows for consistency across repositories. Add notifications on failure. Keep pull request pipelines fast, under ten minutes, by running the heavier tests on merge. Everything lives as code in the repository, reviewed like any other change.

Likely follow-ups

- Why deploy by image digest?
- How do GitHub environments protect production?

```
jobs:
  ci:                          # runs on every PR
    steps:
    - uses: actions/checkout@v4
    - run: make lint test
    - uses: docker/build-push-action@v5
    - uses: aquasecurity/trivy-action@master  # CVE scan
  deploy-staging:                  # runs on merge to main
    needs: ci
    if: github.ref == "refs/heads/main"
    steps:
```

certs.eknathalabs.com

```
    - uses: google-github-actions/auth@v2  # Workload Identity
```

```
    - run: docker push && update GitOps manifests repo
```

```
  deploy-production:              # runs on semver tag
```

```
    if: startsWith(github.ref, "refs/tags/v")
```

```
    environment: production       # manual approval gate
```

PR → CI (lint, test, build, scan) — no deploy. Merge to main → staging deploy automatically. Tag push (v1.2.3) → production with manual approval. Build the image once in the CI job, reference the same digest in deploy jobs. Environment protection rules block production until a required reviewer approves.

📚 **Official docs**

- GitHub Actions documentation

Q6 How do you use dependency caching and matrix builds to speed up pipelines?

GitHub Actions

🎙 **Spoken answer · ~84 s**

Pipeline speed comes from not repeating work and from doing things in parallel. Dependency caching stores downloaded packages, such as node_modules, Maven or pip caches, between runs, keyed by a hash of the lockfile, so the cache is reused when dependencies are unchanged and refreshed when they change. In GitHub Actions, use the cache action or the built-in caching in setup actions. Also cache build outputs and Docker layers, using Buildx cache backends. Be careful about cache poisoning and size limits, and use restore keys for partial hits. Matrix builds run the same job across multiple combinations, such as operating systems, language versions, or test shards, in parallel, so total time is the slowest combination, not the sum. This gives broad compatibility coverage and also lets you shard a long test suite across several runners. Control cost and noise with fail-fast settings, excluding unneeded combinations, and limiting maximum parallelism. Additional speed-ups: run only affected tests or jobs based on changed paths, cancel superseded runs with concurrency groups, use larger or faster runners for heavy steps, and split slow jobs. Measure before and after: track cache hit rates and job durations. Faster feedback keeps developers productive, and it encourages running CI frequently.

Likely follow-ups

- How should a dependency cache key be built?
- How does sharding tests across a matrix shorten the pipeline?

```
strategy:
```

```
  matrix:
```

certs.eknathalabs.com

```
    node: [18, 20, 22]
  fail-fast: false
# Dependency caching:
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${{ hashFiles("**/package-lock.json") }}
    restore-keys: npm-
# Docker layer caching:
- uses: docker/build-push-action@v5
  with:
    cache-from: type=gha
    cache-to:   type=gha,mode=max
```

Matrix builds run N jobs simultaneously — test 3 Node versions in the same CI time as one. Cache key = hash of lock files — invalidated only when dependencies change. Docker layer cache reuses unchanged layers. These two optimisations alone typically reduce pipeline time by 70-80% for Node.js or Python projects.

> 📚 **Official docs**
>
> - GitHub Actions: Workflow syntax
> - GitHub Actions: Caching dependencies

certs.eknathalabs.com

Q7 How does Google Cloud Build work — how do you write a cloudbuild.yaml?

Cloud Build

🎙 Spoken answer · ~85 s

Cloud Build is Google Cloud's managed CI/CD service that runs builds as a series of steps, each in its own container. You define the build in cloudbuild.yaml, with a steps list. Each step has a name, which is the container image to run, such as gcr.io/cloud-builders/docker or any public image, plus args, an optional entrypoint, environment variables, and ids and waitFor to control ordering and parallelism. Steps share a /workspace directory, so outputs from one are visible to the next. Typical file: a step to run tests, a step to docker build with the tag using the substitution variable for the commit SHA, a step to push, and a step to deploy using gcloud or kubectl. The images field lists images to push to the registry on success. Substitutions allow variables like the project ID, short SHA, or user-defined ones with an underscore prefix. Triggers connect repositories, so pushes, pull requests, or tags start builds. Builds run as a service account, which should be a dedicated, least-privilege account. Options include machine type, logging, and timeouts, and secrets come from Secret Manager through the availableSecrets field. Use private pools for VPC access. Cache layers with kaniko or the docker cache-from option to speed builds.

Likely follow-ups

- How do steps share files between each other?
- How does Cloud Build access secrets?

```
steps:
```

```
- name: gcr.io/cloud-builders/docker
```

```
  args: [build, -t, IMAGE:$SHORT_SHA, .]
```

```
- name: gcr.io/cloud-builders/docker
```

```
  args: [push, IMAGE:$SHORT_SHA]
```

```
- name: gcr.io/cloud-builders/gke-deploy
```

```
  args: [run, --filename=k8s/, --cluster=prod, --location=us-central1]
```

```
options: {machineType: E2_HIGHCPU_8, logging: CLOUD_LOGGING_ONLY}
```

Steps run sequentially (use waitFor for parallel). Built-in substitutions: $PROJECT_ID, $SHORT_SHA (7-char commit SHA), $BRANCH_NAME, $TAG_NAME. Triggers connect to GitHub repos on push, PR, or tag. The Cloud Build service account needs IAM permissions for every GCP resource it accesses (Artifact Registry, GKE, Secret Manager).

certs.eknathalabs.com

📚 **Official docs**

- Cloud Build

Q8 How do you integrate Cloud Build with Artifact Registry and Binary Authorization?

Cloud Build

🎙 **Spoken answer · ~83 s**

The integration forms a secure supply chain on Google Cloud. In cloudbuild.yaml, build the image and push it to Artifact Registry, using the repository's regional path, with the build's service account granted Artifact Registry Writer. Cloud Build automatically produces provenance for builds, which is SLSA build attestation, and records image digests. Enable vulnerability scanning on the registry, so images are analysed on push and results are stored as metadata. Binary Authorization then enforces policy at deploy time: you define a policy for a GKE cluster or Cloud Run that requires images to be attested by trusted attestors. The pipeline creates an attestation after tests and scans pass, signing the image digest with a key in Cloud KMS, using the attestation step or Cloud Build's integration. At deployment, the admission controller checks that the image has the required attestations and rejects any that don't, so unverified or manually built images can't run. Policies can also allow-list specific registries, and use dry-run mode first, plus break-glass annotations for emergencies with audit logging. Deploy by digest rather than tag. Combine with VPC Service Controls and IAM least privilege. The outcome is that only images built and verified by your pipeline can reach production.

Likely follow-ups

- What does Binary Authorization check at deploy time?
- Why deploy by digest in this setup?

```
# Build and push to Artifact Registry:
```

```
- name: gcr.io/cloud-builders/docker
```

```
  args: [push, us-central1-docker.pkg.dev/$PROJECT_ID/repo/app:$SHORT_SHA]
```

```
# Container Analysis scans automatically on push
```

```
# Attest after scan passes:
```

```
- name: gcr.io/cloud-builders/gcloud
```

```
  args: [container, binauthz, attestations, sign-and-create,
```

```
         --artifact-url, DIGEST, --attestor, vuln-scan-passed, --keyversion, KMS_KEY]
```

```
# Binary Authorization on GKE rejects unattested images
```

certs.eknathalabs.com

Supply chain: Cloud Build → push to Artifact Registry → Container Analysis auto-scans → if no critical CVEs, sign image digest with KMS key → attestation created → GKE admission controller verifies attestation before running pod. Unsigned images are rejected at the cluster level. This is SLSA Level 3 for GCP-native pipelines.

📚 **Official docs**

- Binary Authorization
- Cloud Build
- Artifact Registry

Q9 How does ArgoCD implement GitOps — what is the sync loop?

ArgoCD/GitOps

🎙 **Spoken answer · ~83 s**

ArgoCD is a GitOps controller for Kubernetes. Git is the source of truth for desired state, in the form of manifests, Helm charts, or Kustomize overlays. You define an Application resource that points to a repository path and target cluster and namespace. The sync loop works like this: ArgoCD's repo server renders the manifests from Git; the application controller continuously compares that desired state with the live state in the cluster, by default every three minutes or on webhook triggers; if they differ, the application is marked OutOfSync. With a manual sync policy, an operator reviews and triggers sync; with automated sync, ArgoCD applies the changes itself. Options include prune, which removes resources deleted from Git, and self-heal, which reverts manual changes made in the cluster, correcting drift. Sync uses waves and hooks for ordering, such as running a migration job before the rollout, and tracks health status for each resource, so you see Healthy, Progressing, or Degraded. Benefits: auditability through Git history, easy rollback by reverting a commit, drift detection, and the cluster pulling changes so credentials need not live in CI. Practices: separate application source from configuration repositories, protect branches, use RBAC and SSO, and monitor sync status.

Likely follow-ups

- What do prune and self-heal do?
- How is rollback done in GitOps?

certs.eknathalabs.com

```
argocd app create myapp --repo https://github.com/org/manifests \
```

```
  --path apps/myapp/overlays/prod \
```

```
  --sync-policy automated --self-heal --prune
```

```
argocd app sync myapp          # manual sync
```

```
argocd app diff myapp          # what would change
```

```
argocd app rollback myapp 3    # rollback to revision 3
```

ArgoCD continuously reconciles cluster state with the Git manifests repo. Every 3 minutes (or on webhook) it compares desired state (Git) with actual state (cluster). auto-sync=true → applies changes automatically. self-heal=true → reverses manual kubectl changes. prune=true → deletes resources removed from Git. Result: Git is the only way to change cluster state. Rollback = git revert in the manifests repo.

📚 **Official docs**

- OpenGitOps principles
- Argo CD documentation

Q10 **What is the App-of-Apps pattern — how do you bootstrap a cluster with ArgoCD?** ArgoCD/GitOps

🎙 **Spoken answer · ~84 s**

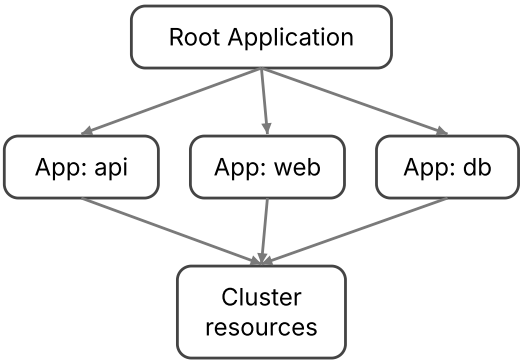
The App-of-Apps pattern uses one ArgoCD Application whose source is a directory of other Application manifests, so deploying that single root application causes ArgoCD to create and manage all the child applications. This gives a declarative way to bootstrap an entire cluster from Git. Bootstrapping works like this: install ArgoCD in the cluster, by Helm or manifests, as the only manual step; then apply one root Application pointing to a Git path, for example clusters/production/apps. That folder holds Application resources for cluster add-ons, such as ingress controller, cert-manager, monitoring, and external secrets operator, followed by workload applications. Sync waves or ordering annotations ensure that dependencies like CRDs and operators are installed before things that use them. The root application can manage ArgoCD itself, so its configuration is also in Git. Benefits: reproducible clusters, easy disaster recovery by recreating them from the repository, consistent environments, and one place to add a new application. ApplicationSets are an evolution: they generate Applications from templates using generators for clusters, directories, or Git files, which handles multi-cluster and multi-tenant scenarios with less repetition. Practices: structure repositories per environment, use projects to restrict what each team can deploy, and protect the root application's path with strict review.

Likely follow-ups

- What is the single manual step in bootstrapping?
- How do ApplicationSets improve on App-of-Apps?

certs.eknathalabs.com

🗺 **App-of-Apps**




```
# Root app manages child Application CRDs:
```

```
argocd app create root-app --repo https://github.com/org/manifests --path cluster/apps
```

```
cluster/apps/
```

```
  monitoring.yaml    # Application → monitoring stack
```

```
  ingress.yaml       # Application → ingress-nginx
```

```
  myapp.yaml         # Application → myapp manifests
```

```
# Apply root-app once → ArgoCD manages everything else
```

```
# ApplicationSet for multi-cluster/environment generation
```

Bootstrap a new cluster: apply the root app, ArgoCD installs everything else. Entire cluster config is in Git — any new cluster is bootstrapped identically. ApplicationSet generates Applications dynamically from templates (one template for all environments, or all clusters in a fleet). This is how platform teams manage dozens of clusters with a single GitOps repo.

📚 **Official docs**

- Kubernetes: Troubleshooting kubeadm
- Argo CD documentation

certs.eknathalabs.com

Q11 How do you handle secrets in GitOps — what integrates with ArgoCD?

ArgoCD/GitOps

🎙 Spoken answer · ~82 s

The core challenge is that GitOps stores everything in Git, but secrets must not be in plain text there. Several approaches integrate with ArgoCD. External Secrets Operator: Git holds only an ExternalSecret resource referencing a secret in Vault, AWS Secrets Manager, GCP Secret Manager, or Azure Key Vault; the operator in the cluster fetches the real value and creates a Kubernetes Secret. Nothing sensitive is in Git, rotation is handled at the source, and ArgoCD simply syncs the reference. Sealed Secrets: you encrypt a secret with the cluster's public key into a SealedSecret custom resource that is safe to commit; only the controller in the cluster can decrypt it. SOPS with age or KMS keys encrypts values in files, decrypted by a plugin such as KSOPS or helm-secrets during rendering. The Secrets Store CSI driver mounts secrets directly into pods from a provider. HashiCorp Vault with the agent injector or the Argo CD Vault plugin is another route. My preference is External Secrets Operator, because secrets live in a proper manager with auditing and rotation. Whichever you choose, restrict access with RBAC, avoid ArgoCD displaying secret values, protect the decryption keys, and have a rotation procedure.

Likely follow-ups

- Why prefer External Secrets over Sealed Secrets?
- How do rotated secrets reach running pods?

```
# External Secrets Operator (recommended for GCP):
```

```
kubectl get externalsecret db-password  # syncs Secret Manager → K8s Secret
```

```
# Sealed Secrets (encrypt in Git):
```

```
kubeseal --fetch-cert --controller-name sealed-secrets
```

```
kubectl create secret generic db --from-literal=pass=mysecret \
```

```
  --dry-run=client -o yaml | kubeseal -o yaml > sealed-secret.yaml
```

```
git add sealed-secret.yaml  # safe to commit encrypted
```

```
# Vault Agent Injector:
```

```
vault.hashicorp.com/agent-inject: "true"  # injects secret as pod file
```

Never commit plain Kubernetes Secrets (base64 is not encryption). External Secrets Operator: secrets in GCP Secret Manager, ESO syncs into K8s Secrets — never in Git. Sealed Secrets: encrypted in Git, cluster-specific key, simpler ops. Vault: most powerful, most complex, handles dynamic credentials. For GCP + GitOps: ESO + Secret Manager is the production standard.

certs.eknathalabs.com

📚 **Official docs**

- OpenGitOps principles
- Mozilla SOPS
- Argo CD documentation

Q12 What types of tests belong in CI — how do you keep pipelines under 10 minutes?

Testing

🎙 **Spoken answer · ~82 s**

CI should contain tests that give fast, reliable feedback on every change, arranged like a pyramid. Static analysis comes first: linting, formatting, type checks, and secret and dependency scans, taking seconds. Unit tests form the base: many, fast, isolated, running in parallel. Integration tests verify components together, such as the service with a real database in a container, but are fewer. Contract tests check API compatibility between services without a full environment. A small number of end-to-end tests cover critical user journeys, often run after merge or in a staging environment rather than on every commit, since they are slow and flakier. Security scans, container image scans, and performance smoke tests also fit. To keep pipelines under ten minutes: run jobs in parallel and shard test suites; cache dependencies and Docker layers; run only affected tests based on changed files; fail fast by ordering cheap checks first; use faster runners for heavy steps; eliminate or quarantine flaky tests; avoid redundant builds; and split a fast pull request pipeline from a slower full pipeline on merge or nightly. Measure stage durations regularly and treat pipeline speed as a product. Slow pipelines get bypassed and reduce deployment frequency.

Likely follow-ups

- What belongs in the fast pull request pipeline versus the post-merge pipeline?
- How do you handle flaky tests?

```
Unit tests:        < 2 min — every commit (fast, isolated)
```

```
Integration tests: 5-10 min — against real dependencies (DB, cache)
```

```
Contract tests:    < 5 min — API contracts between services
```

```
E2E / smoke:       10-20 min — post-deploy on staging (not blocking PR)
```

```
# Speed techniques:
```

```
pytest -x -q -n auto    # parallel, fail-fast
```

```
jest --findRelatedTests  # only tests for changed files
```

```
docker compose up -d db redis  # real deps for integration
```

certs.eknathalabs.com

Keep PR pipeline under 10 minutes — developers abandon longer pipelines. Parallelize tests (pytest-xdist, jest --maxWorkers). Fail fast (-x). Split slow E2E into a post-merge pipeline. The test pyramid: many unit, fewer integration, minimal E2E.

📚 Official docs

- GitHub Actions: Security hardening (pipeline as attack vector)

Q13 How do you implement shift-left security in a CI/CD pipeline?

Security

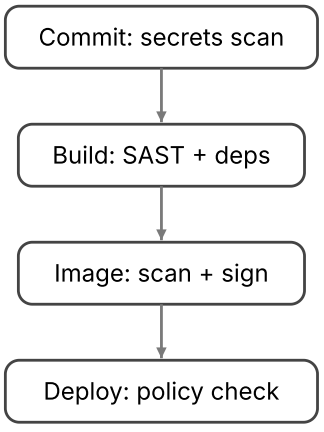
🎙 Spoken answer · ~83 s

Shift-left security means finding and fixing issues early, when they're cheapest, by embedding checks in the pipeline and developer workflow. Layers: in the IDE and pre-commit, secret detection and linting; on pull requests, static application security testing with tools such as Semgrep or CodeQL, software composition analysis for vulnerable dependencies and licences, secret scanning, and infrastructure-as-code scanning with Checkov or tfsec; at build, container image scanning with Trivy, SBOM generation, and Dockerfile linting; before deploy, image signing and policy checks, such as admission policies requiring signed images; and after deploy, dynamic testing (DAST) against staging, and runtime monitoring. Set gates sensibly: fail builds on critical and high findings with fixes available, report lower severities, and allow tracked exceptions with expiry. Avoid noise, because noisy scanners get ignored; tune rules, and prioritise by exploitability. Make results visible in pull requests as comments or annotations, so developers see them in context. Pin dependencies, use a private artefact mirror, and protect the pipeline itself, with least-privilege tokens and pinned actions. Provide secure templates and golden paths so security is the default. Track metrics such as mean time to remediate, and educate developers. The goal is to make secure practice fast and frictionless.

Likely follow-ups

- Why does noise undermine shift-left security?
- What gates would you set on findings?

🗺 Shift-left security



certs.eknathalabs.com

```
# Pre-commit: detect-secrets, lint, basic SAST

# CI pipeline:

trivy image myapp:$SHA --exit-code 1 --severity CRITICAL   # container CVE scan

npm audit --audit-level=high                               # dependency audit

checkov -d terraform/                                      # IaC misconfiguration

tflint --recursive                                         # Terraform lint

# GitHub Advanced Security (GHAS):

- uses: github/codeql-action/analyze@v3                    # SAST
```

Shift-left: find security issues earlier when cheaper to fix. Pre-commit (seconds), CI (minutes), staging (hours), production (expensive). Failing CI on CRITICAL CVEs prevents known vulnerabilities from reaching production. IaC scanning catches misconfigured infrastructure before it is deployed. Cost of a security fix grows 10x at each stage.

> 📚 **Official docs**
>
> - OWASP DevSecOps Guideline

certs.eknathalabs.com

Q14 How do you design a multi-environment pipeline with promotion gates?

Pipeline design

🎙 Spoken answer · ~81 s

A multi-environment pipeline promotes a single immutable artefact through increasingly production-like stages. Stages typically include dev, staging, and production, ideally in separate accounts or projects for isolation. Build once in CI, publish the versioned artefact, then deploy the same artefact to each environment with environment-specific configuration injected separately. Promotion gates decide when it moves on. Automated gates: unit and integration tests passed, security scans clean, deployment health checks green, smoke tests, performance baselines, and monitoring checks after a bake period. Manual gates: approvals by a release manager or product owner where regulation or risk requires, implemented as environment protection rules. Restrict who can approve, and require that staging succeeded and that the artefact digest matches. Production deployment uses progressive strategies, canary or blue-green, with automatic rollback on alarm. Keep environments structurally similar, using infrastructure as code, so staging results predict production. Handle database changes with backward-compatible migrations. Use concurrency controls so only one deployment per environment runs at a time. Maintain an audit log of who approved and deployed what. Make promotion visible on a dashboard showing which version runs where. Keep gates meaningful and fast; too many manual gates slow delivery without adding safety.

Likely follow-ups

- Why keep staging structurally similar to production?
- Which gates should be automatic versus manual?

```
PR open    → CI (lint, test, scan) — no deployment
```

```
Merge main → auto-deploy dev    → smoke tests
```

```
Tag v*     → auto-deploy staging → integration tests → manual gate
```

```
Approve    → deploy production  → smoke → monitor 30min
```

```
# GitHub Actions environment protection:
```

```
environment: production
```

```
  protection_rules:
```

```
    required_reviewers: [tech-lead, sre-team]
```

Build once, promote the same image through environments. Each gate validates the artifact is safe for the next stage. Dev: fast feedback. Staging: full integration + performance baseline. Production: manual approval + automated canary. If staging tests fail, the artifact does not proceed. The promotion decision is based on data (test results, metrics), not opinion.

certs.eknathalabs.com

📚 **Official docs**

- GitHub Actions: Environments & deployment protection

Q15 How do you handle database migrations in a CI/CD pipeline?

Pipeline design

🎙 **Spoken answer · ~88 s**

Database migrations are risky in CI/CD because schema and application versions must work together during a rollout. The principle is backward-compatible changes using expand and contract: first expand the schema in a way old code tolerates, such as adding a nullable column or new table; deploy the application that uses it; backfill data; and only later contract by removing old structures once nothing uses them. Never combine a destructive change with the deploy that depends on it. Run migrations as a distinct, controlled step in the pipeline, before the application rollout, using a versioned tool like Flyway, Liquibase, or the framework's migrations, with the migration files in the repository. Run them once, via a job or pipeline stage with a lock, not from every application instance at startup. Test migrations in CI against a real database, and in staging against production-like data to measure duration and locking. Use online techniques for large tables: batched backfills, concurrent index creation, and lock timeouts. Use credentials with the privileges needed only for migration. Take a backup or snapshot before risky changes, and make rollback plans: since schema rollbacks are hard, rely on compatible changes so the application can roll back independently. Monitor the database during the migration, and halt the pipeline on failure.

Likely follow-ups

- Why run migrations as a separate pipeline step?
- What is the expand and contract pattern?

```
# Init container (migration before new pods start):
initContainers:
- name: migrate
  image: myapp:$VERSION
  command: ["python", "manage.py", "migrate"]
# Pre-deploy Kubernetes Job:
kubectl apply -f migration-job.yaml
kubectl wait --for=condition=complete job/db-migration --timeout=5m
kubectl apply -f deployment.yaml   # only after migration succeeds
```

certs.eknathalabs.com

```
# Expand-contract for zero-downtime migrations:
```

```
1. Add nullable column  2. Deploy (writes both)  3. Backfill
```

```
4. Deploy (reads new)   5. Drop old column
```

Never run migrations in app startup — multiple replicas race. Init containers or pre-deploy Jobs run once, sequentially. Expand-contract ensures both old and new app versions work with the same schema simultaneously — required for rolling deployments. Always test migration on a production data snapshot in staging first.

📚 **Official docs**

- Flyway: Database migrations in CI/CD

Q16 How do you implement rollback in a CI/CD pipeline?

Pipeline design

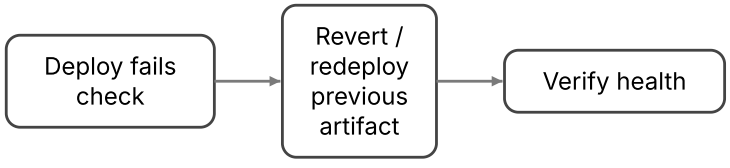
🎙 **Spoken answer · ~81 s**

Rollback should be fast, automated, and rehearsed. The foundation is immutable, versioned artefacts: rolling back means deploying a previous known-good artefact, not rebuilding. Mechanisms depend on the platform. In Kubernetes, kubectl rollout undo reverts a Deployment to the previous ReplicaSet, and helm rollback restores a previous release. With GitOps, revert the commit in the configuration repository and let ArgoCD or Flux reconcile, keeping Git as the truth. With blue-green, switch traffic back to the old environment; with canary, abort the rollout and send all traffic to the stable version. Feature flags provide the fastest rollback: turn the flag off without redeploying. Automate triggers: define health checks and alerts, such as error rate or latency thresholds after deployment, so the pipeline or rollout controller reverts automatically. In the pipeline, include a manual rollback job or one-click redeploy of a previous version, and keep recent artefacts available. Consider data: database migrations should be backward compatible so the old code works with the new schema; irreversible changes need a forward-fix plan. Test rollbacks regularly, measure time to restore, and keep the process documented. After a rollback, investigate root cause in a non-production environment and run a blameless review.

Likely follow-ups

- Why is a feature flag the fastest rollback?
- How do database migrations complicate rollback?

🗺 **Rollback**



certs.eknathalabs.com

```
# Fast: kubectl rollout undo (seconds)
```

```
kubectl rollout undo deployment/myapp
```

```
kubectl rollout undo deployment/myapp --to-revision=3
```

```
# Safe: GitOps revert (auditable, goes through pipeline)
```

```
git revert HEAD --no-edit && git push origin main  # ArgoCD auto-syncs
```

```
# Automated: Argo Rollouts analysis template
```

```
kubectl argo rollouts abort myapp      # manual abort
```

```
# Redeploy previous tag:
```

```
kubectl set image deployment/myapp myapp=registry/myapp:v1.2.2
```

GitOps revert is the production standard — creates audit trail, goes through the normal pipeline, tests run, visible to the team. Automated rollback (Argo Rollouts AnalysisTemplate): if error rate crosses threshold within 5 minutes of deploy, rollback automatically with no human required. This is the highest-maturity deployment safety mechanism.

> 📚 **Official docs**
>
> - Kubernetes: Deployments
> - AWS Builders' Library: Ensuring rollback safety during deployments

certs.eknathalabs.com

Q17 How do you monitor ArgoCD and alert on sync failures?

ArgoCD/GitOps

🎙 Spoken answer · ~83 s

Monitoring ArgoCD covers both the controller's health and the state of the applications it manages. ArgoCD exposes Prometheus metrics from its components: the application controller, API server, and repo server. Key metrics: argocd_app_info, with labels for sync status and health status, lets you alert when an application is OutOfSync for too long or Degraded; sync operation results and failures; reconciliation duration and queue depth, to detect a slow controller; repo server errors and Git fetch latency; and Kubernetes API request errors. Scrape them with a ServiceMonitor, and visualise them in Grafana using the community dashboards. Alerting rules: an application Degraded for more than a few minutes; OutOfSync for longer than expected, which can indicate drift or a failed sync; sync failures; repository connection errors; and the controller being down. Route alerts through Alertmanager to Slack or PagerDuty with the application name and a link. ArgoCD Notifications is a built-in controller that sends messages on events such as sync failed, sync succeeded, or health degraded, to Slack, email, or webhooks, with templates, and can post deployment status to Git providers. Also monitor the cluster-level view, tracking many applications across clusters. Create runbooks for common failures like invalid manifests or webhook issues.

Likely follow-ups

- Which ArgoCD metrics would you alert on?
- What is ArgoCD Notifications used for?

```
# Key Prometheus alerts:
```

```
argocd_app_health_status{health_status="Degraded"} > 0
```

```
argocd_app_sync_status{sync_status="OutOfSync"} > 0
```

```
argocd_app_sync_total{phase="Failed"}  # rate increasing
```

```
# Check app status:
```

```
argocd app list
```

```
argocd app get myapp
```

```
kubectl get applications -n argocd -o wide
```

Alert on Degraded health (app not running), OutOfSync for more than 30 minutes (sync may have failed silently), and increasing sync failure rate. ArgoCD exposes a Prometheus metrics endpoint. Integrate ArgoCD notifications to Slack for deployment success/failure messages in the team channel — this replaces manual deploy status checking.

certs.eknathalabs.com

📚 **Official docs**

- Argo CD documentation

Q18 CI passes but deployments fail in production — how do you debug the gap?

Troubleshooting

🎙 **Spoken answer · ~82 s**

When CI passes but production fails, the gap is some difference between the CI environment and production, so I look for what differs. Compare configuration: environment variables, feature flags, secrets, connection strings, and resource limits. Compare data: production data volume, shape, and edge cases that test data lacks, such as nulls, large payloads, or legacy records. Compare infrastructure: versions of the runtime, base image, database, Kubernetes, network policies, IAM permissions, and certificates. Compare scale and concurrency: load-related issues like race conditions, connection pool exhaustion, and timeouts that only appear under traffic. Check dependencies: external services, rate limits, DNS, and proxies absent in test. Check the artefact: confirm that the deployed artefact is the one that was tested, by digest. Check the deployment process itself: migration ordering, partial rollouts, missing permissions, or readiness probes. Then use evidence: logs, traces, and events from the failing deployment, and reproduce locally or in staging with production-like configuration. To close the gap long term: make staging mirror production through infrastructure as code, run smoke and integration tests post-deploy, use canary releases with automated analysis, test with realistic data, shift observability earlier, and track the failure as a missing test or check.

Likely follow-ups

- What kinds of differences most often explain CI-versus-production failures?
- How do you close the gap permanently?

```
# Verify correct image is deployed:
```

```
kubectl get deployment myapp -o jsonpath=".spec.template.spec.containers[0].image"
```

```
# Check env vars in production:
```

```
kubectl exec myapp-pod -- env | sort
```

```
# Check rollout events:
```

```
kubectl describe deployment myapp | grep -A10 Conditions
```

```
kubectl get events --sort-by=.lastTimestamp -n production | tail -20
```

```
# Pod logs after deploy:
```

certs.eknathalabs.com

```
kubectl logs -l app=myapp --since=10m | grep -i error
```

Common gaps: secrets present in CI but missing in production (not created as K8s Secrets), resource limits too low (CI runs without limits), network policies blocking production connections, readiness probe timeout too short (production dependencies are slower than CI). Check each systematically rather than guessing.

📚 **Official docs**

- GitHub Actions: Security hardening (pipeline as attack vector)

Q19 A CI pipeline takes 45 minutes — how do you find and fix the bottleneck? Troubleshooting

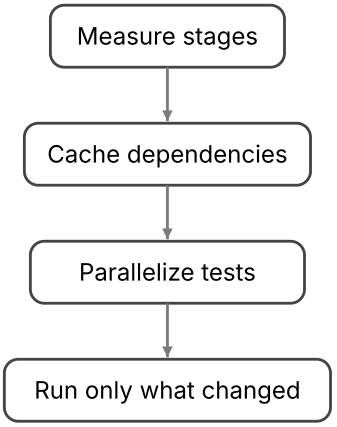
🎙 **Spoken answer · ~84 s**

I treat a slow pipeline as a performance problem: measure first. Collect timing data per stage and job, and look at the critical path, the longest sequence of dependent jobs, since only that determines total time. Typical bottlenecks and fixes. Dependency installation: use caching keyed by lockfiles. Docker builds: order Dockerfile layers for caching, use multi-stage builds, and remote cache backends. Tests: the slowest part usually; parallelise by sharding across runners, run only affected tests, move slow end-to-end tests out of the pull request path, and fix or remove slow and flaky tests. Serial jobs: restructure so independent jobs run in parallel instead of in sequence. Waiting for runners: queue time can dominate, so scale up the runner pool or use autoscaling runners. Resource-starved runners: use larger machines for the heavy steps. Redundant work: avoid rebuilding unchanged components in a monorepo with path filters or build graph tools. Large checkouts: shallow clones and sparse checkouts. Network: use a registry mirror or local cache for artefacts. Set a time budget, such as ten minutes for pull requests, track it as a metric with alerts when it regresses, and review regularly. Go after the biggest wins first, verify improvements, and prevent regression by monitoring.

Likely follow-ups

- How do you identify the critical path of a pipeline?
- What are the first fixes for a slow test stage?

🗺 **Speeding up CI**



certs.eknathalabs.com

```
# Check step timing in GitHub Actions workflow run UI
# Most common bottlenecks:
1. No caching: npm install / pip install every run (5-15 min)
2. Docker build without layer cache (10-20 min)
3. Sequential jobs that could be parallel
4. Full test suite when only affected tests needed
5. Shallow clone not used (full git history download)
# Fixes:
actions/cache@v4 for dependencies
cache-from: type=gha for Docker layers
fetch-depth: 1 for shallow clone
```

Profile before optimising — look at GitHub Actions step timings. Usually one step accounts for 80% of the time. Dependency caching alone often reduces pipeline from 20 minutes to 3 minutes. Parallelise independent jobs (lint + test + security scan run simultaneously). The 10-minute target for PR pipelines is achievable for most services with proper caching.

> 📚 **Official docs**
>
> - GitHub Actions: Caching dependencies

certs.eknathalabs.com

Q20 How do you use reusable workflows and composite actions for pipeline standardisation?

GitHub Actions

🎙 Spoken answer · ~83 s

Reusable workflows and composite actions let you define pipeline logic once and use it across many repositories. A reusable workflow is a complete workflow with a workflow_call trigger, defined with inputs, secrets, and outputs; other repositories call it with uses and a reference to a version, like org/repo/.github/workflows/build.yml@v2. It contains whole jobs, so it's ideal for standardising an entire stage, such as build, scan, and publish, with consistent policies. A composite action packages a sequence of steps into a single action, with inputs and outputs, defined in an action.yml; it is used within a job as a step and suits smaller building blocks, like setting up tools with caching, or running a standard scan. Use reusable workflows for job-level orchestration and composite actions for step-level reuse. Standardisation benefits: consistent security checks, fewer copy-paste errors, central updates, and faster onboarding. Practices: version them with tags and pin consumers to a major version, or a commit SHA for security; document inputs; keep them in a central repository with access control; test them with their own pipeline; and avoid over-parameterisation that makes them hard to use. Use organisation-level required workflows or rulesets to enforce baseline checks. Provide starter templates for new repositories.

Likely follow-ups

- When do you choose a composite action over a reusable workflow?
- How do you version shared workflows safely?

```
# Reusable workflow (.github/workflows/deploy.yaml):
on:
  workflow_call:
    inputs:
      environment: {type: string, required: true}
    secrets:
      GCP_WIF_PROVIDER: {required: true}
# Caller:
jobs:
  deploy:
    uses: ./.github/workflows/deploy.yaml
```

certs.eknathalabs.com

```
    with: {environment: production}
```

```
    secrets: inherit
```

Reusable workflows: define deploy-to-gke once, call it from 10 service repos. Composite actions: bundle multiple steps (setup-gcp, setup-docker) into one action. For a platform team managing many service repos, reusable workflows enforce pipeline standards without managing each repo individually. Teams get consistent CI/CD by convention.

📚 **Official docs**

- GitHub Actions: Reusable workflows

Q21 What is a feature flag — how does it decouple deployment from release?

Pipeline design

🎙 **Spoken answer · ~85 s**

A feature flag is a conditional in code controlled by configuration at runtime, so behaviour can be switched on or off, or targeted to specific users, without deploying new code. It decouples deployment, putting code into production, from release, exposing a feature to users. Code ships dark, behind a flag that's off; later, product or engineering turns it on for internal users, then a percentage, then everyone. Benefits: trunk-based development, since unfinished features can merge safely behind flags; instant rollback by turning off the flag, with no redeploy; progressive rollouts and canary releases by user segment; A/B testing and experimentation; and kill switches for risky dependencies. Flags come in types: release flags, short-lived; ops flags or kill switches; experiment flags; and permission flags, long-lived. Management matters, as flags carry costs: complexity from branching code paths, testing combinations, and flag debt when old flags are never removed. Practices: name flags clearly with owners and expiry dates, remove them after rollout, keep flag evaluation fast and resilient with sensible defaults if the flag service is down, log flag states in observability data, avoid nesting flags, and use a managed service like LaunchDarkly or an open-source one such as Unleash. Protect access to flag changes in production.

Likely follow-ups

- What is flag debt and how do you control it?
- What default should apply if the flag service is unavailable?

```
# Simple env var flag:
```

```
if os.getenv("ENABLE_NEW_PAYMENT_FLOW") == "true":
```

```
    return new_payment_flow(request)
```

```
return legacy_payment_flow(request)
```

coriseKnathalabs.com

```
# Managed (LaunchDarkly / GrowthBook / Flagsmith):
```

```
if ld_client.variation("new-payment-flow", user, False):
```

```
    return new_payment_flow(request)
```

```
# Gradual rollout: 1% users → 10% → 50% → 100%
```

```
# Instant rollback: flip flag off — no deployment needed
```

Feature flags decouple deployment (code in production) from release (users see it). Merge incomplete features behind a flag — enables trunk-based development without long-lived branches. Gradual rollout to 1%, monitor metrics, increase. Instant rollback: disable the flag in seconds, no deployment required. Also enables A/B testing and canary releases without Kubernetes traffic splitting complexity.

📚 **Official docs**

- Martin Fowler: Feature Toggles

Q22 How do you implement contract testing between microservices?

Testing

🎙 **Spoken answer · ~85 s**

Contract testing verifies that services can communicate by checking the agreement between a consumer and a provider, without running both together in a full environment. In consumer-driven contract testing, using a tool such as Pact, the consumer team writes tests describing the requests it makes and the responses it expects; running those generates a contract file. The contract is published to a broker. The provider team then verifies the contract against its real implementation in its own pipeline, replaying the consumer's requests and checking that the responses match. If a provider change would break a consumer's expectations, the provider's build fails before release. Benefits over end-to-end testing: fast, reliable, and independent of the whole system running; early feedback; and a clear record of who depends on what. Practices: keep contracts focused on what the consumer really uses, not the entire API; run consumer tests in the consumer's CI, publish with the version and branch; run provider verification on every provider build and on contract change via webhooks; use the broker's can-i-deploy check as a gate before deployment, to confirm compatible versions; and handle provider states for setup data. Contract tests complement, not replace, schema validation with OpenAPI and a small number of end-to-end tests.

Likely follow-ups

- What does can-i-deploy check?
- How do contract tests differ from end-to-end tests?

```
# Pact: consumer-driven contract testing
```

certs.eknathalabs.com

```
# Consumer defines expected interactions:
```

```
pact.given("user exists").upon_receiving("GET /users/123")
```

```
    .will_respond_with(200, body={"id":123, "name":"Alice"})
```

```
# Provider verifies it satisfies all consumer contracts:
```

```
pact verify --provider=payments-api --pact-url=pact-broker/contracts.json
```

Contract tests catch API breaking changes before deployment without needing both services running. Much faster than E2E tests. The consumer defines what it expects, the provider verifies it can satisfy those expectations. Essential in microservice architectures where teams develop services independently. Catches "provider broke consumer without knowing" — the most common integration failure mode.

📚 **Official docs**

- Pact: Contract testing

Q23 How do you prevent secrets from leaking in CI/CD pipeline output?

Security

🎙 **Spoken answer · ~87 s**

Preventing leaks in pipeline output starts with treating logs as public. Use the CI system's secret masking: GitHub Actions masks registered secrets, and other platforms have similar features, but masking is pattern-based and can be bypassed by transformations like base64 encoding or splitting, so don't rely on it alone. Never echo secrets or print environment dumps; avoid set -x in scripts that handle credentials; and don't pass secrets as command-line arguments, which can appear in process listings and logs. Reduce exposure: use short-lived credentials via OIDC rather than stored keys, and inject secrets only into the steps that need them, not the whole job. Don't expose secrets to untrusted code, such as pull requests from forks, and avoid pull_request_target patterns that run untrusted code with privileges. Configure tools to suppress verbose output: Docker build args shouldn't carry secrets; use build secrets mounts instead, since arguments persist in image history. Scan logs and artefacts: run secret scanners over build outputs, uploaded artefacts, and images. Limit log retention and access. Set up alerts for secret scanning detections, and have a rapid rotation process for anything exposed. Pin third-party actions to avoid malicious ones exfiltrating secrets. Review pipeline changes carefully, because modifying a workflow can be a path to leak secrets.

Likely follow-ups

- Why isn't log masking enough on its own?
- How should Docker builds receive secrets safely?

```
# Mask secrets immediately in GitHub Actions:
```

certs.eknathalabs.com

```
echo "::add-mask::$SECRET"   # masks in all subsequent output
```

```
# Never echo secrets:
```

```
echo $SECRET                 # BAD — visible in logs
```

```
# Docker build: use --secret (not build args):
```

```
docker build --secret id=token,env=GITHUB_TOKEN .  # not in image history
```

```
# Scan logs for accidental exposure:
```

```
gitleaks detect --source . --verbose
```

Common leak vectors: echo in debug scripts, error messages printing full environment, curl with Authorization headers in verbose mode, Docker build args (visible in image history). Use --secret flag for sensitive values in Docker builds — they do not persist in image layers. Use Workload Identity Federation to eliminate static credential files entirely — nothing to leak.

📚 **Official docs**

- Docker documentation

Q24 How do you measure CI/CD pipeline performance — what are the key metrics?

Fundamentals

🎙 **Spoken answer · ~85 s**

Pipeline performance is measured at two levels: the delivery outcomes and the pipeline's own mechanics. The DORA metrics capture delivery performance: deployment frequency, lead time for changes from commit to production, change failure rate, and time to restore service. Within the pipeline, I track: build and test duration overall and per stage, with percentiles, especially the p50 and p95 time for pull request checks; queue time waiting for runners; success rate and the reason for failures, distinguishing real failures from infrastructure problems; flaky test rate, meaning tests that fail then pass on retry; cache hit rates; cost per build; and the time to feedback for developers. Also useful: how long a pull request waits for review and merge, the mean time to recover a broken main branch, and the percentage of deployments rolled back. Collect from the CI platform's API or export into a dashboard, and set targets and alerts when metrics regress. Look at trends, not single runs. Use the numbers to prioritise improvements: if queue time dominates, add runners; if tests dominate, shard or prune. Avoid using the metrics to rank individuals, since this encourages gaming. Share dashboards with teams and review them regularly, treating the pipeline as a product with users.

Likely follow-ups

- Which DORA metrics measure delivery performance?
- Why measure queue time separately from run time?

certs.eknathalabs.com

```
# DORA metrics:

Deployment frequency:   how often do you deploy to production?

Lead time for changes:  commit to production time (hours vs days)

Change failure rate:    % of deploys causing an incident

MTTR:                   how fast do you recover from failure?

# Pipeline-specific metrics:

PR cycle time           PR open → merge

CI pass rate            % of pipeline runs that succeed

Pipeline P95 duration   95th percentile build time

Cache hit rate          % of builds using cached deps
```

Elite teams: deploy multiple times/day, <1 hour lead time, <5% change failure rate, <1 hour MTTR. If lead time is days, the bottleneck is usually manual approval gates, slow test suites, or large PRs. Track these in a dashboard and treat pipeline improvements as engineering work, not housekeeping.

> 📚 **Official docs**
>
> - DORA: The four key metrics

certs.eknathalabs.com

Q25 How do you debug a flaky CI test — passes locally, fails intermittently in CI?

Troubleshooting

🎙 Spoken answer · ~86 s

A test that passes locally but fails intermittently in CI points to nondeterminism or an environmental difference. First, gather evidence: how often it fails, which job or runner, and the error. Rerun it many times in a loop in CI, or with the failing seed, to reproduce and capture logs, screenshots, or traces. Look for the common causes. Timing and asynchrony: sleeps, race conditions, and waiting on elements or services that aren't ready; replace fixed delays with explicit waits for conditions. Shared state and order dependence: tests that share databases, files, or global variables, and interfere when run in parallel or in a different order; isolate state per test and randomise order locally to expose dependencies. Resource differences: CI runners with less CPU or memory, causing timeouts; slow I/O. Environment differences: time zones, locale, file system case sensitivity, and dependency versions. External dependencies: network calls to real services; mock them or use containers with health checks. Non-deterministic data: random values, dates, and ordering of unordered collections. Fixes: remove the nondeterminism, don't just add retries. In the interim, quarantine the test with a ticket and track flaky rates, so it doesn't block everyone, and make flakiness visible. Prevent recurrence with review guidelines and tools that detect flaky tests.

Likely follow-ups

- Why shouldn't you just add retries to a flaky test?
- How do you find order-dependent tests?

```
# Common causes:
```

```
1. Timing (sleep() instead of wait_until())
```

```
2. Port conflicts in parallel test runs
```

```
3. Shared mutable state between tests (test pollution)
```

```
4. External service dependencies (network timeouts)
```

```
5. Resource contention on CI runner (CPU throttling)
```

```
# Debug: run the test 10 times in isolation
```

```
pytest tests/test_payment.py -v --count=10
```

```
# Replace sleep with polling:
```

```
wait_until(lambda: service.ready(), timeout=30)
```

```
@pytest.mark.flaky(reruns=3)   # short-term quarantine
```

certs.eknathalabs.com

Flaky tests erode trust in CI — engineers start ignoring failures. Track flakiness rate per test (failures/runs), prioritise high-flakiness tests. Short-term: retry logic. Long-term: fix root cause. Never leave a flaky test untracked — it will eventually mask a real failure at the worst possible time.

📚 **Official docs**

- Google Testing Blog: Flaky tests at Google

Q26 How do you optimise Cloud Build for speed and cost?

Cloud Build

🎙 **Spoken answer · ~84 s**

Optimising Cloud Build means reducing both build time and the billed minutes. For speed: cache layers, using Kaniko with the cache flag or Docker's cache-from with a previously pushed image; cache dependencies by storing them in a Cloud Storage bucket or building a base image with dependencies pre-installed; run independent steps in parallel using waitFor with '-', instead of sequentially; use a larger machine type for heavy compilation, which costs more per minute but often less overall; use multi-stage Dockerfiles and small base images so pushes and pulls are quick; and keep source uploads small with a .gcloudignore file. Use private pools when you need VPC access or dedicated capacity, and keep builds in the same region as Artifact Registry to reduce latency and egress. For cost: choose the smallest machine that meets the time target; avoid unnecessary builds through trigger filters that include only relevant paths and exclude documentation changes; cancel superseded builds; set sensible timeouts; clean up old artefacts and logs with retention policies; and make use of the free tier minutes. Consider whether to run lightweight checks elsewhere. Monitor build durations and costs per trigger, and review regularly. Fast, efficient builds keep feedback quick and the bill predictable.

Likely follow-ups

- How do trigger filters reduce cost?
- How do you parallelise Cloud Build steps?

```
# Faster machine type:
```

```
options: {machineType: E2_HIGHCPU_8}   # 8 vCPU, 4x cost but 6-8x faster
```

```
# Docker layer caching via Artifact Registry:
```

```
- name: gcr.io/cloud-builders/docker
```

```
  args: [build, --cache-from, IMAGE:latest, -t, IMAGE:$SHORT_SHA, .]
```

```
# Parallel steps:
```

```
- id: test
```

```
  waitFor: [build]
```

certs.eknathalabs.com

```
- id: scan
```

```
  waitFor: [build]  # runs in parallel with test
```

Cloud Build billing: per build-minute x machine type. E2_HIGHCPU_8 is typically net-cheaper for CPU-bound builds (6-8x faster, only 4x more expensive). Parallelise steps with waitFor. Docker layer caching from Artifact Registry significantly reduces build time. Private worker pools let builds access VPC resources (Cloud SQL, private GKE) without public exposure.

📚 **Official docs**

- Cloud Build
- GitHub Actions: Caching dependencies

Q27 How do you use ApplicationSets for multi-cluster deployments?

ArgoCD/GitOps

🎙 **Spoken answer · ~85 s**

ApplicationSets are an ArgoCD feature that generates multiple Applications from a single template using generators. Instead of writing one Application per cluster or environment, you define an ApplicationSet with a template and one or more generators, and the controller creates, updates, and deletes the Applications as the generator output changes. Common generators: the list generator, with explicit values; the cluster generator, which iterates over clusters registered in ArgoCD, optionally selected by labels, perfect for deploying a baseline to every cluster; the Git directory generator, which creates an Application per folder in a repository; the Git file generator, using config files; the matrix and merge generators to combine, for example clusters multiplied by applications; and the pull request generator for preview environments. For multi-cluster deployments: register the clusters in ArgoCD with labels such as environment and region, define an ApplicationSet that targets clusters by label, and templating parameters like the cluster name and URL fill into each Application. Roll out gradually using progressive sync strategies, updating dev clusters first, then staging, then production. Benefits: less repetition, consistent configuration, and automatic onboarding of new clusters. Practices: use ArgoCD projects to constrain what ApplicationSets can create, review changes carefully since they affect many clusters, and monitor generation errors.

Likely follow-ups

- What does the cluster generator do?
- How do you roll out gradually across clusters with ApplicationSets?

```
apiVersion: argoproj.io/v1alpha1
```

```
kind: ApplicationSet
```

```
spec:
```

certs.eknathalabs.com

```
  generators:
  - clusters: {}    # all registered clusters
  template:
    spec:
      source:
        path: apps/myapp/overlays/{{name}}
      destination:
        server: {{server}}
```

ApplicationSet generates N ArgoCD Applications from one template — one per cluster, environment, or Git branch. Cluster generator deploys to every registered cluster automatically. Git generator creates an app per subdirectory. Matrix generator combines both. This scales GitOps to 100+ clusters with a single template definition. New cluster registration → automatic app deployment → zero manual configuration.

> 📚 **Official docs**
>
> - Argo CD: ApplicationSet
> - Argo CD documentation

certs.eknathalabs.com

Q28 What is pipeline as code — how do you version and test it?

Pipeline design

🎙 Spoken answer · ~85 s

Pipeline as code means defining the build and deployment process in version-controlled files, such as GitHub Actions YAML, a Jenkinsfile, or cloudbuild.yaml, stored alongside the application. The benefits are the same as for any code: review through pull requests, history and blame, rollback, reuse, and consistency across branches, with no snowflake configurations hidden in a UI. Versioning: keep pipeline definitions in the repository, so each commit is built with the pipeline that matches it; for shared logic, put it in versioned libraries, reusable workflows, or templates, with tags and semantic versions, and pin consumers to versions so updates are deliberate. Testing: lint and validate syntax, for example with actionlint or the platform's validation tools; test shared components in their own repository with example consumers; run pipeline changes on a branch or a sandbox repository before merging to main; use dry runs where the tool supports them; and keep scripts in separate files that can be tested locally, with shell checks, instead of embedding long scripts in YAML. Apply review rules and protected branches, because changing a pipeline is changing a path to production. Use least-privilege tokens and secrets scoping, and audit changes. Keep pipelines simple, declarative, and documented, and avoid logic duplicated across repositories.

Likely follow-ups

- How do you test a change to a pipeline safely?
- Why version shared pipeline templates?

🗺 Pipeline as code



certs.eknathalabs.com

```
# Pipeline config lives in the application repo:
.github/workflows/ci.yaml   # GitHub Actions
cloudbuild.yaml              # Cloud Build
# Test locally:
act -j test                  # run GitHub Actions locally (act tool)
cloud-build-local --dryrun cloudbuild.yaml
# Lint pipeline files:
actionlint .github/workflows/*.yaml
# Pipeline changes go through PR review like code
```

Pipeline as code: CI/CD config in the same repo as the application. Changes go through PR review. Prevents pipeline drift across teams. Enables rollback (git revert the pipeline change). Provides audit trail. Test on a feature branch before merging — open a test PR, verify the pipeline behaves as expected. Changes to the pipeline are as significant as changes to the application code.

> 📚 **Official docs**
>
> - Martin Fowler: Deployment Pipeline

certs.eknathalabs.com

Q29 How do you implement canary deployments in a CI/CD pipeline?

Pipeline design

🎙 Spoken answer · ~88 s

A canary deployment sends a small share of traffic to the new version, observes its behaviour, and then gradually increases exposure, aborting if it looks unhealthy. In a pipeline, the stages are: build and publish an immutable artefact; deploy the new version alongside the stable one at low weight; shift traffic by a small percentage, for example five percent; hold and analyse metrics for a defined period; then progress to twenty-five, fifty, and one hundred percent if healthy; otherwise roll back automatically. Implementation options: Kubernetes with Argo Rollouts or Flagger, which manage ReplicaSets, traffic splitting through an ingress controller or service mesh, and automated analysis; load balancer weighted target groups; CodeDeploy canary configurations on AWS; Cloud Deploy or Cloud Run traffic splitting on Google Cloud. Analysis compares canary and baseline on error rate, latency, saturation, and business metrics, from Prometheus or a monitoring service, with explicit thresholds. Requirements: good observability with metrics labelled by version, enough traffic to produce a meaningful signal, backward-compatible database changes, and health checks. Decide how sessions and users are routed consistently. Define abort criteria and automate rollback. Notify the team of progress, and have a manual promote or abort control for sensitive releases. Canary reduces blast radius, and is a safer default than all-at-once for user-facing services.

Likely follow-ups

- What metrics should the canary analysis compare?
- Why do you need enough traffic for a canary to be meaningful?

```
# Argo Rollouts canary strategy:
spec:
  strategy:
    canary:
      steps:
      - setWeight: 10    # 10% traffic to new version
      - pause: {duration: 5m}
      - analysis:        # check error rate
          templates: [{templateName: error-rate}]
      - setWeight: 50
      - pause: {duration: 10m}
```

certs.eknathalabs.com

```
      - setWeight: 100
```

```
kubectl argo rollouts get rollout myapp --watch
```

Canary: route a small percentage of traffic to the new version, measure golden signals, progressively increase if healthy, automatically rollback if metrics degrade. Argo Rollouts AnalysisTemplate queries Prometheus: if error rate exceeds threshold during the canary phase, rollback automatically. This limits blast radius — a bad deploy affects only 10% of users before automatic rollback. The metrics-driven promotion is the key advantage over simple rolling updates.

📚 **Official docs**

- Google SRE Workbook: Canarying Releases
- Argo Rollouts
- Martin Fowler: CanaryRelease

Q30 Design a complete CI/CD platform for 20 engineers across 10 microservices on GCP.

Pipeline design

🎙 **Spoken answer · ~87 s**

For twenty engineers and ten microservices on GCP, I'd design a simple, secure, standardised platform and avoid over-engineering. Source: GitHub or Cloud Source Repositories with trunk-based development, pull request checks, and branch protection. CI: Cloud Build triggers, or GitHub Actions with Workload Identity Federation to GCP, running tests, linting, security scans, and building container images with caching. A shared pipeline template, with reusable steps, keeps services consistent. Artefacts: Artifact Registry with vulnerability scanning, immutable tags, and image provenance; Binary Authorization to require attested images on GKE or Cloud Run. CD: GitOps with Argo CD or Flux for GKE, with a config repository per environment, or Cloud Deploy for managed promotion across dev, staging, and production, with approvals; progressive delivery with canary and automatic rollback. Environments: separate projects per environment with infrastructure as code through Terraform, VPC Service Controls where needed, and least-privilege service accounts. Secrets: Secret Manager with workload identity, no static keys. Quality: contract tests between services, feature flags, and post-deploy smoke tests. Observability: Cloud Monitoring and Logging, Managed Prometheus, tracing, and SLOs with burn-rate alerts. Developer experience: service templates, documentation, and a fast pipeline target under ten minutes. Measure DORA metrics. Start with a thin slice for two services, learn, then roll out to the rest.

Likely follow-ups

- Why GitOps or Cloud Deploy for the CD layer?
- How would you roll this platform out to ten services?

Principal-level platform design question — architecture, developer experience, and governance.

certs.eknathalabs.com

```
Source:     GitHub + branch protection + CODEOWNERS + signed commits
CI:         GitHub Actions reusable workflows (canonical pipeline per language)
Registry:   GCP Artifact Registry + Container Analysis auto-scan
CD:         ArgoCD + two-repo GitOps (app repo / manifests repo)
Secrets:    External Secrets Operator + GCP Secret Manager
Security:   Binary Authorization + Workload Identity
Preview:    Ephemeral environments per PR (short-lived GKE namespace)
Metrics:    DORA dashboard (deploy freq, lead time, CFR, MTTR)
```

Platform team owns: reusable workflow templates, manifests repo structure, ArgoCD ApplicationSet config, DORA metrics dashboard. Service teams only write application code — CI/CD is inherited by using the platform Dockerfile conventions and reusable workflows. Goal: a new service is production-deployable within 1 day of creation, using the same pipeline as all other services. Platform SLO: commit to production in under 30 minutes including manual approval.

> **Developer experience first:** A fast, reliable pipeline is the platform team's product. Track pipeline P95 duration and treat improvements as feature work.

> 📚 **Official docs**
>
> - Google Cloud Build + Cloud Deploy (CI/CD on GCP)

certs.eknathalabs.com

Q31 What is trunk-based development and why does it suit CI/CD?

Fundamentals

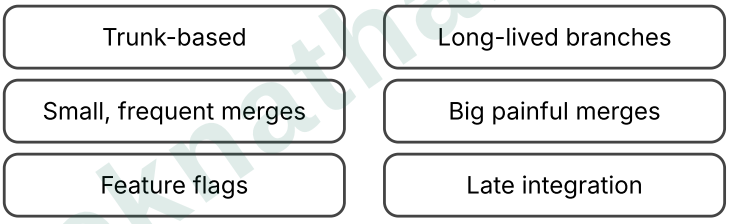
🎙 Spoken answer · ~58 s

Trunk-based development means everyone integrates into a single main branch very frequently, ideally at least daily, using short-lived branches that live hours rather than weeks. It suits CI/CD because continuous integration only works if changes are actually integrated continuously. Long-lived feature branches hide conflicts and create big, risky merges, and they make the pipeline test something that never resembles production. With trunk-based work, unfinished functionality is hidden behind feature flags so main is always releasable. It requires strong automated tests, fast pipelines and small pull requests with quick reviews. Compared with Gitflow, it removes release branches and cherry-pick chaos, though some teams keep short release branches for hotfixes. The benefits are smaller diffs, easier reviews, faster feedback and more frequent deployment. The trade-off is that the team needs discipline and good test coverage, otherwise broken changes reach main more often.

Likely follow-ups

- How do feature flags fit in?
- When would you still use release branches?

🗺 Trunk vs long branches



Developers merge small changes to one main branch at least daily, using short-lived branches and feature flags. It keeps integration continuous and avoids long-lived branch merge pain.

```
git switch -c fix/login-timeout
```

```
git pull --rebase origin main
```

```
git push origin fix/login-timeout
```

Interview tip: Say that flags, not branches, hide unfinished work.

📚 Official docs

- Trunk Based Development

certs.eknathalabs.com

Q32 What are the DORA metrics and how do they relate to CI/CD?

Fundamentals

🎙 Spoken answer · ~60 s

DORA metrics are four measures of software delivery performance: deployment frequency, lead time for changes, change failure rate and time to restore service. The first two describe throughput and the last two describe stability. CI/CD directly influences them. A fast, reliable pipeline shortens lead time from commit to production and enables more frequent deployments. Automated tests, progressive delivery and quick rollback lower the change failure rate and speed up recovery. I would measure them from real data, such as pipeline events and incident records, not from estimates, and look at trends per team instead of ranking people. The research finding is that high performers are good at both speed and stability, so they are not a trade-off. I use the metrics to find bottlenecks, for example long review waits or slow test stages, and I avoid turning them into targets that people could game.

Likely follow-ups

- How would you collect these metrics?
- Why avoid using them as targets?

Deployment frequency, lead time for changes, change failure rate and time to restore service. Together they measure delivery speed and stability.

```
git log --merges --since=30.days --oneline | wc -l
```

```
kubectl rollout history deploy/app
```

Interview tip: Emphasise that speed and stability are measured together.

📚 Official docs

- DORA: The four key metrics

certs.eknathalabs.com

Q33 What is the difference between a build artifact and a deployment manifest, and why version both? Fundamentals

🎙 **Spoken answer · ~59 s**

A build artifact is the output of the build, such as a container image, a JAR or a binary, identified by an immutable version or digest. A deployment manifest describes how to run it: Kubernetes YAML, a Helm chart with values, or Terraform that defines the runtime. I version both because a release is the combination of the two. If I only version the artifact, I cannot reproduce configuration, replica counts, resource limits or environment settings from last week. If I only version the manifest, it may point at a mutable tag that now refers to different code. Storing manifests in Git gives history, review and easy rollback, and referencing artifacts by digest makes the deployment deterministic. Many teams keep manifests in a separate repository from application code so that configuration changes can be reviewed and promoted without rebuilding the artifact.

Likely follow-ups

- Why use image digests?
- Why separate config from code repositories?

The artifact is the built binary or image; the manifest says how and where to run it. Versioning both gives reproducible, auditable releases.

```
docker build -t app:1.4.2 .
```

```
helm template app ./chart --set image.tag=1.4.2
```

Interview tip: Mention that rollbacks need both to be recoverable.

📚 **Start here (official docs)**

- GitHub Actions documentation

certs.eknathalabs.com

Q34 What is a monorepo versus polyrepo pipeline strategy, and how does CI handle each?

Fundamentals

🎙 Spoken answer · ~57 s

In a monorepo, many services and libraries live in one repository. The CI challenge is avoiding building and testing everything on every change. I use path filters or a build tool with a dependency graph, such as Bazel, Nx or Pants, to calculate affected targets and cache results remotely. The advantages are atomic cross-service changes, shared tooling and easy refactoring. In a polyrepo setup, each service has its own repository and pipeline, which gives clear ownership and simple pipelines, but cross-repository changes need coordination, version bumps and compatibility testing. For a monorepo I would also make sure code ownership files, permissions and pipeline times scale, since a single slow pipeline blocks everyone. The right choice depends on team size, coupling between services and tooling maturity, and I would not choose a monorepo without investing in incremental builds.

Likely follow-ups

- How do you find affected services?
- What breaks at monorepo scale?

A monorepo holds many services; CI must detect which changed. Polyrepo has one pipeline per repo but harder cross-repo changes.

```
git diff --name-only origin/main...HEAD | cut -d/ -f1 | sort -u
```

Interview tip: Talk about path filters and build caching.

📚 Start here (official docs)

- GitHub Actions documentation

certs.eknathalabs.com

Q35 What are the stages of a typical modern CI pipeline in order, and why that order?

Fundamentals

🎙 Spoken answer · ~58 s

I order stages from cheapest and fastest to most expensive, so failures are reported early and compute is not wasted. First come static checks: formatting, linting and type checks, plus secret scanning. Next, unit tests, which are fast and need no infrastructure. Then the build of the artifact, followed by security scanning of dependencies and the image. After that, integration tests against ephemeral dependencies, and finally publishing the immutable artifact to a registry, tagged with the commit SHA. Deployment pipelines then promote that same artifact through environments. Some stages can run in parallel, such as linting and unit tests, to reduce wall-clock time. The principle is fail fast: developers should learn about a typo in a minute, not after a thirty-minute end-to-end suite. I also make sure every stage is deterministic and that the pipeline definition lives in version control.

Likely follow-ups

- Which stages can run in parallel?
- Where do end-to-end tests belong?

Lint and unit tests first, then build, scan, integration tests, publish artifact. Cheap fast checks come first to fail early.

```
make lint test
```

```
docker build -t app:$SHA .
```

```
trivy image app:$SHA
```

Interview tip: Explain fail-fast and cost ordering.

📚 Start here (official docs)

- GitHub Actions documentation

certs.eknathalabs.com

Q36 What is a build agent or runner, and how do you choose between hosted and self-hosted?

Fundamentals

🎙 Spoken answer · ~60 s

A runner or build agent is the machine or container that executes pipeline jobs. Hosted runners are managed by the provider, start clean for each job and need no maintenance, which is ideal for small teams and public code. Self-hosted runners give control over hardware, such as GPUs or large memory, access to private networks and internal services, and often lower cost at high volume. The risks are security and upkeep. A long-lived self-hosted runner can retain state or secrets from previous jobs and can be attacked through untrusted pull requests. I prefer ephemeral runners that are created per job, for example on Kubernetes or autoscaling groups, with minimal permissions and no access to production credentials unless the job needs them. I would also never run untrusted fork code on runners with privileged network access, and I monitor queue time to scale capacity.

Likely follow-ups

- Why prefer ephemeral runners?
- How do you secure runners for forked pull requests?

A runner executes pipeline jobs. Hosted is simple and clean; self-hosted gives control, private network access and cost savings at scale.

```
docker run --rm -it ubuntu:24.04 bash
```

Interview tip: Cover isolation, ephemeral runners and secrets.

📚 Start here (official docs)

- GitHub Actions documentation

certs.eknathalabs.com

Q37 What is semantic versioning and how do you automate releases from commits?

Fundamentals

🎙 Spoken answer · ~64 s

Semantic versioning uses MAJOR.MINOR.PATCH, where a patch is a backwards-compatible fix, a minor adds backwards-compatible functionality, and a major introduces breaking changes. It communicates risk to consumers. To automate releases I use conventional commit messages, such as feat, fix and a breaking change footer, and a tool like semantic-release or release-please that reads commits since the last tag, calculates the next version, generates a changelog, creates a tag and publishes the artifact. This removes manual version decisions and keeps history consistent. It works best for libraries and APIs. For internal services deployed continuously, many teams version by commit SHA or build number instead, since consumers do not need compatibility signals. The pipeline should only publish from the main branch, require passing checks, and protect the release credentials. I review the setup regularly so it keeps matching how the team really works. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- How is a breaking change signalled?
- When is SHA tagging better?

MAJOR.MINOR.PATCH conveys compatibility. Conventional commit messages let tools calculate the next version and changelog automatically.

```
git commit -m 'feat: add retry to client'
```

```
npx semantic-release --dry-run
```

Interview tip: Mention breaking change markers.

📚 Official docs

- Semantic Versioning

certs.eknathalabs.com

Q38 How do you restrict permissions of the GITHUB_TOKEN in a workflow?

GitHub Actions

🎙 Spoken answer · ~58 s

Every workflow run receives a token that can act on the repository, and by default its scope can be broader than needed. I set a top-level permissions block to the minimum, usually contents read, and then grant extra scopes only in the jobs that need them, for example pull-requests write for a commenting job or id-token write for cloud authentication with OpenID Connect. I also set the organisation or repository default to read-only so a workflow that forgets the block is still safe. This limits the damage if a malicious dependency or a compromised action runs in the job, since the token cannot push code or change releases. Combined with pinning third-party actions to commit SHAs and protecting environments with required reviewers, it forms a solid baseline. I review permissions in code review like any other access control change.

Likely follow-ups

- Why default to read-only?
- Which jobs need id-token write?

Set permissions at the workflow or job level to the minimum scopes and default to read-only.

```
permissions:
  contents: read
  id-token: write
```

Interview tip: Least privilege per job.

📚 Start here (official docs)

- GitHub Actions documentation

certs.eknathalabs.com

Q39 How do you authenticate a workflow to AWS, Azure or GCP without long-lived secrets?

GitHub Actions

🎙 Spoken answer · ~56 s

Instead of storing long-lived access keys as repository secrets, I use OpenID Connect federation. The workflow requests a signed identity token from the CI provider, containing claims such as repository, branch, environment and workflow. In the cloud, I create an identity provider trust and a role or workload identity that accepts tokens only when those claims match, for example the main branch of one repository and a production environment. The workflow exchanges the token for short-lived credentials that expire in about an hour. This removes secret rotation and leakage risk, and gives a clear audit trail. The common mistake is a trust policy that is too broad, such as any repository in the organisation, so I always constrain subject claims. I also separate roles per environment so a pull request build cannot deploy to production.

Likely follow-ups

- What claims should the trust policy check?
- What are the failure modes?

Use OpenID Connect federation: the workflow gets a short-lived token the cloud trusts based on repository, branch and environment claims.

```
permissions:
  id-token: write
  contents: read
```

Interview tip: Scope the trust policy to branch or environment.

📚 Official docs

- GitHub Actions: OpenID Connect

certs.eknathalabs.com

Q40 Why pin third-party actions to a commit SHA, and how do you keep them updated?

GitHub Actions

🎙 Spoken answer · ~57 s

A tag or branch reference in a workflow can be moved by the action's maintainer, or by an attacker who compromises their account, so a previously safe step can start running malicious code in my pipeline with access to secrets. Pinning to a full commit SHA makes the reference immutable. The downside is maintenance, since SHAs are unreadable and updates are manual. I solve that with a dependency update bot configured for workflows, which opens pull requests that bump the SHA and show the release notes and diff, so a human reviews each change. I add a comment with the version tag for readability. For critical workflows I also prefer actions maintained by the platform or by my organisation, fork important third-party actions into an internal repository, and restrict the allowed actions list at the organisation level.

Likely follow-ups

- What is an allowed-actions policy?
- How do update bots help?

Tags can be moved; a SHA is immutable. Use an update bot to propose reviewed bumps.

```
uses: actions/checkout@<full-commit-sha> # v4.1.1
```

Interview tip: Supply chain risk.

📚 Start here (official docs)

- GitHub Actions documentation

certs.eknathalabs.com

Q41 How do you design environment protection and approvals for production deployments?

GitHub Actions

🎙 Spoken answer · ~58 s

I model each deployment target as an environment, such as staging and production, and attach protection rules. For production I require reviewers from a specific team, restrict deployments to the main branch or protected tags, and optionally add a wait timer. Secrets and variables are scoped to the environment so a job only sees production credentials after approval. This gives a clear audit trail of who approved what and when. I keep the deploy job separate from the build job, so the same artifact built and tested earlier is promoted, not rebuilt. To avoid approval fatigue, I limit manual gates to production and risky changes, and automate checks like smoke tests and health verification after deployment. For emergencies there should be a documented break-glass path with extra logging, so urgent fixes do not push people to bypass the system.

Likely follow-ups

- Why scope secrets to environments?
- How do you handle emergency deploys?

Use environments with required reviewers, wait timers, branch restrictions and environment-scoped secrets.

```
environment: production
```

```
needs: [build, test]
```

Interview tip: Gate on environment, not on person.

📚 Official docs

- GitHub Actions: Environments & deployment protection

certs.eknathalabs.com

Q42 How do concurrency groups prevent overlapping deployments?

GitHub Actions

🎙 Spoken answer · ~61 s

Concurrency groups let me declare that only one run in a given group can execute at a time. For pull request validation I set the group to the branch and enable cancel in progress, so a new push cancels the outdated run and saves compute. For deployments I use a group per environment and set cancel in progress to false, so runs queue in order and a deployment is never killed halfway, which could leave the system in a partial state. This prevents two pipelines applying infrastructure or migrations at the same time, which can corrupt state or cause race conditions. Combined with state locking in the infrastructure tool, it provides defence in depth. One caveat is that pending runs in a group may be replaced by a newer one, so for strict ordering I design deploys to be idempotent and always deploy the latest commit.

Likely follow-ups

- When should you cancel in-progress runs?
- What happens to pending runs?

A concurrency key cancels or queues runs sharing the same group so two deploys do not race.

```
concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: false
```

Interview tip: Cancel for CI, queue for deploy.

📚 Official docs

- GitHub Actions: Workflow syntax

certs.eknathalabs.com

Q43 How do you build and publish a container image securely with provenance in a workflow?

GitHub Actions

🎙 Spoken answer · ~56 s

I build the image using a pinned builder in a clean runner, tag it with the commit SHA and push it, then work with the digest. I enable build provenance and generate a software bill of materials so consumers can see how and from what the image was built. Next I scan the image for vulnerabilities and fail on critical findings with a documented exception process. Then I sign the image digest with a tool like Cosign, preferably keyless using the workflow's identity, so the signature proves it came from my pipeline. In the cluster, an admission policy verifies the signature and refuses unsigned images. The key idea is signing the digest, not a mutable tag. This chain gives traceability from running container back to commit and workflow, which helps with audits and incident response.

Likely follow-ups

- Why sign digests instead of tags?
- What does admission verify?

Build with a pinned builder, sign the image, generate an SBOM and attach provenance attestations.

```
docker buildx build --provenance=true --sbom=true -t reg/app:$SHA --push .
```

```
cosign sign reg/app@sha256:...
```

Interview tip: Sign the digest, not the tag.

📚 Official docs

- GitHub: Artifact attestations (build provenance)

certs.eknathalabs.com

Q44 How do you reduce cost and time of workflows that run on every pull request?

GitHub Actions

🎙 **Spoken answer · ~56 s**

I start by measuring which jobs consume the most minutes, then apply targeted fixes. Path filters skip workflows when only documentation changes. Concurrency groups with cancel in progress stop outdated runs after a new push. Dependency and build caches reduce repeated downloads, and I key them on lock file hashes. I trim the test matrix on pull requests to the most important combinations and run the full matrix on main or nightly. Large jobs are split and parallelised, and slow tests are sharded. I use the smallest runner that meets the need and skip expensive jobs for draft pull requests, with a label to opt in. I also set timeouts so stuck jobs do not burn minutes. Finally, I review the bill monthly and track pipeline duration as a metric so regressions are noticed.

Likely follow-ups

- How do you key a cache?
- What should run only on main?

Use path filters, caching, concurrency cancel, smaller matrices and skip drafts.

```
on:
  pull_request:
    paths: ['src/**','package.json']
```

Interview tip: Measure before optimising.

📚 **Start here (official docs)**

- GitHub Actions documentation

certs.eknathalabs.com

Q45 What is the danger of pull_request_target and how do you handle workflows for forks?

GitHub Actions

🎙 **Spoken answer · ~56 s**

The pull_request event from a fork runs with a read-only token and no secrets, which is safe but limited. The pull_request_target event runs in the context of the base repository with secrets and a write token available. If a workflow using it checks out and executes code from the fork, an attacker can submit a pull request that exfiltrates secrets or pushes changes. My rule is never to run untrusted code in a privileged context. For tasks like labelling or commenting I use pull_request_target without checking out fork code. For building and testing forks I use the plain pull_request event, and if secrets are needed I use a two-stage design: an unprivileged workflow produces artifacts, and a separate privileged workflow consumes them as data only. I also require approval before workflows run for first-time contributors.

Likely follow-ups

- How do two-stage workflows work?
- Why require approval for new contributors?

It runs with secrets in the base repository context; checking out untrusted fork code there can leak secrets.

```
on: pull_request
```

```
# avoid checkout of PR head under pull_request_target
```

Interview tip: Never run untrusted code with secrets.

📚 **Start here (official docs)**

- GitHub Actions documentation

certs.eknathalabs.com

Q46 How do you use substitutions and variables in Cloud Build safely?

Cloud Build

🎙 **Spoken answer · ~57 s**

Cloud Build supports built-in substitutions such as project ID, commit SHA, short SHA, branch and tag, and user-defined substitutions that must start with an underscore. I use them so a single configuration file serves multiple environments, for example an environment name and region passed from the trigger, instead of copying files. Values can be set in the trigger definition or on the command line. For security, I never pass secrets as substitutions, because they appear in logs and build metadata. Secrets come from Secret Manager through the available secrets section, which injects them as environment variables only for the steps that request them. In shell steps I need to escape the dollar sign so Cloud Build does not try to substitute shell variables. I also validate substitution values in an early step to fail fast on typos.

Likely follow-ups

- How are secrets injected?
- How do you escape shell variables?

Built-in substitutions like $SHORT_SHA and user-defined ones starting with an underscore let one config serve many environments.

```
gcloud builds submit --substitutions=_ENV=staging,_REGION=us-central1
```

```
# cloudbuild.yaml: ${_ENV}
```

Interview tip: User substitutions must start with underscore.

📚 **Official docs**

- Cloud Build

certs.eknathalabs.com

Q47 How do you run Cloud Build in a private pool and why?

Cloud Build

🎙 Spoken answer · ~55 s

The default Cloud Build workers run in a Google-managed network with public internet access. A private pool is a dedicated worker pool connected to my VPC through private peering, so builds can reach internal resources such as private Git servers, artifact repositories, databases for integration tests or on-premises systems over VPN. It also allows choosing machine types and disk sizes and can disable public IPs, which satisfies strict network policy and data exfiltration controls. The trade-offs are cost, since pools are billed for the workers' use, and extra setup of peering and firewall rules. I use private pools when a build needs private connectivity or compliance requires controlled egress. For ordinary public builds, the default pool is simpler and cheaper. I also combine private pools with VPC Service Controls to limit data movement.

Likely follow-ups

- What are the cost trade-offs?
- How do you control egress?

Private pools run builds in a VPC-connected, customer-owned environment, allowing access to private resources and custom machine sizes.

```
gcloud builds worker-pools create ci-pool --region=us-central1 --peered-
network=projects/p/global/networks/n
```

Interview tip: Use for private repos and databases.

📚 Official docs

- Cloud Build

certs.eknathalabs.com

Q48 How do you speed up Docker builds in Cloud Build using caching?

Cloud Build

🎙 Spoken answer · ~64 s

Cloud Build workers are ephemeral, so Docker layer cache does not persist between builds by default. I restore cache in a few ways. One is pulling the previous image and using cache-from, so unchanged layers are reused. Another is using Kaniko with a remote cache repository in Artifact Registry, which stores layers explicitly, or BuildKit with registry cache export. Equally important is Dockerfile ordering: copy dependency manifests and install dependencies first, then copy source code, so the expensive layer is invalidated only when dependencies change. I also use multi-stage builds and a small base image, add a dockerignore file to avoid cache busting from irrelevant files, and pin base image versions. These changes typically reduce build time from minutes to under a minute for incremental changes. The aim is a pipeline people trust enough to ship from on a Friday afternoon. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- Why is cache lost on ephemeral workers?
- What is Kaniko?

Use Kaniko or BuildKit cache, pull the previous image as cache-from, and order Dockerfile layers by change frequency.

```
docker build --cache-from $IMAGE:latest -t $IMAGE:$SHORT_SHA .
```

Interview tip: Cache is lost between ephemeral workers.

📚 Official docs

- Cloud Build
- Docker documentation

certs.eknathalabs.com

Q49 How do you deploy to Cloud Run from Cloud Build with a gradual rollout?

Cloud Build

🎙 Spoken answer · ~58 s

Cloud Run creates an immutable revision on every deploy, which makes progressive delivery straightforward. In the pipeline I deploy the new image with no traffic and a tag, which gives the revision its own URL for smoke tests. If the checks pass, I shift a small percentage of traffic to it, say ten percent, watch error rate and latency for a defined period, then increase to fifty and a hundred percent. Rollback is just pointing traffic back to the previous revision, which takes seconds and needs no rebuild. I keep the previous revisions available and set the pipeline to automate the checks, using monitoring queries to decide whether to promote or revert. For configuration changes, I include them in the revision so the entire release is atomic, and I make database migrations backwards compatible so both revisions can run together.

Likely follow-ups

- How do you test the new revision before traffic?
- How do you roll back?

Deploy a new revision with no traffic, test it using a tag URL, then shift traffic in stages and roll back by re-pointing traffic.

```
gcloud run deploy app --image $IMG --no-traffic --tag canary
```

```
gcloud run services update-traffic app --to-tags canary=10
```

Interview tip: Revisions are immutable.

📚 Official docs

- Kubernetes: Deployments
- Cloud Build
- Cloud Run

certs.eknathalabs.com

Q50 How do you control who can trigger builds and what the build can access in Cloud Build?

Cloud Build

🎙 Spoken answer · ~57 s

By default, builds run as a service account that historically had broad permissions. I create dedicated user-managed service accounts per pipeline with only the roles needed, for example permission to push to one Artifact Registry repository and deploy one Cloud Run service. Triggers are configured with that service account, so a build for a test project cannot touch production. I restrict who can edit triggers and submit manual builds through IAM, because someone who can change a trigger can effectively run code as its service account. For pull requests from contributors, I use comment-based approval so builds do not run automatically with privileged access. Logs go to a controlled bucket, and audit logs record trigger changes. I periodically review service account roles and remove unused ones, and I separate build and deploy accounts for sensitive environments.

Likely follow-ups

- Why use a custom service account?
- How do you handle pull request builds?

Use a dedicated user-managed service account with least privilege per trigger, and restrict trigger configuration with IAM.

```
gcloud builds triggers create github --service-
account=projects/p/serviceAccounts/ci@p.iam.gserviceaccount.com
```

Interview tip: Avoid the default broad build account.

📚 Official docs

- Cloud Build

certs.eknathalabs.com

Q51 How do you implement build provenance and attestations on Google Cloud? Cloud Build

🎙 Spoken answer · ~66 s

Cloud Build can generate signed provenance for built container images, describing the source repository, commit, builder and build steps, aligned with SLSA levels. It is stored with the image metadata in Artifact Registry. By itself, provenance is only evidence. The value comes from verification. With Binary Authorization on GKE or Cloud Run, I define a policy that requires attestations from trusted attestors, such as one proving the image was built by my official pipeline from the main branch and another proving it passed vulnerability scanning. Deployments of images without valid attestations are blocked. This protects against someone pushing an image built on a laptop or from a compromised source. I also use the break-glass mode with logging for emergencies, and I store the policy in version control so changes are reviewed. I document the decision so new engineers understand why it is built this way. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- What does Binary Authorization check?
- What is the break-glass procedure?

Cloud Build generates SLSA provenance; Binary Authorization verifies attestations before deployment.

```
gcloud artifacts docker images describe $IMG --show-provenance
```

Interview tip: Verify at admission, not just generate.

📚 Official docs

- GitHub: Artifact attestations (build provenance)
- SLSA specification

certs.eknathalabs.com

Q52 How do you handle drift and self-healing in ArgoCD?

ArgoCD/GitOps

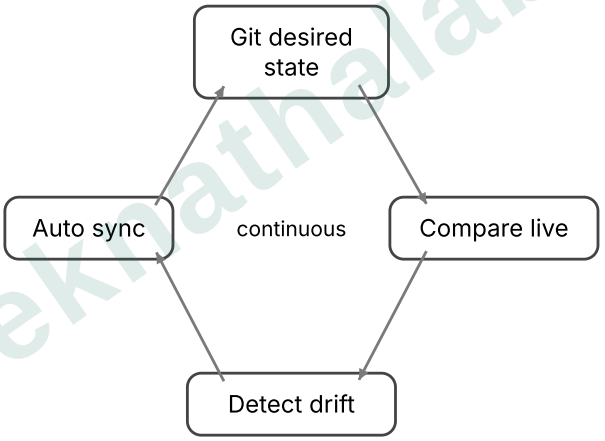
🎙 Spoken answer · ~55 s

Drift occurs when the live cluster differs from what Git declares, for example someone edits a deployment with kubectl. ArgoCD continuously compares them and marks the application out of sync. With automated sync and self-heal enabled, ArgoCD reverts the manual change back to the Git state, and with prune enabled it deletes resources removed from Git. This enforces Git as the single source of truth and helps with audit and security. The trade-off is that during an incident, an engineer's emergency patch will be reverted, so the process must be to commit the fix, or temporarily disable auto-sync on that application with a clear note. Some fields are legitimately changed by controllers, such as replica counts managed by an autoscaler, so I configure ignore differences for those to avoid constant out-of-sync noise.

Likely follow-ups

- What are ignore-differences used for?
- How do you do emergency changes?

🗺 ArgoCD self-heal



Automated sync with selfHeal reverts manual cluster changes to match Git; prune removes resources deleted from Git.

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
```

Interview tip: Self-heal can fight emergency changes.

📚 Official docs

- Terraform CLI: plan
- Argo CD documentation

certs.eknathalabs.com

Q53 What are sync waves and hooks in ArgoCD and when do you use them?

ArgoCD/GitOps

🎙 **Spoken answer · ~65 s**

By default ArgoCD applies resources in a sensible kind order, but sometimes I need explicit ordering. Sync waves are annotations with an integer: resources in lower waves are applied and must become healthy before higher waves begin. I use them to create namespaces and custom resource definitions before the resources that depend on them. Hooks are resources, usually jobs, that run at specific phases: PreSync before the main sync, Sync, PostSync after success, and SyncFail on failure. A typical use is running a database migration job in PreSync and a smoke test in PostSync. Hooks can be cleaned up automatically with delete policies. I keep hooks idempotent and short, because failing hooks block the sync, and I avoid overusing them since complex orchestration inside a sync is hard to debug. I review the setup regularly so it keeps matching how the team really works. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- What is a PreSync hook used for?
- What happens when a hook fails?

Waves order resource creation within a sync; hooks run jobs at phases like PreSync and PostSync.

```
argocd.argoproj.io/sync-wave: "-1"
```

```
argocd.argoproj.io/hook: PreSync
```

Interview tip: Use for migrations and CRD ordering.

📚 **Official docs**

- Argo CD: Sync waves and hooks
- Argo CD documentation

certs.eknathalabs.com

Q54 How do you structure repositories and environments for ArgoCD: Kustomize overlays or Helm values?

ArgoCD/GitOps

🎙 Spoken answer · ~66 s

I prefer a directory-per-environment layout on a single branch, rather than a branch per environment, because branches drift and merges become confusing. With Kustomize there is a base containing common manifests and overlays for dev, staging and production that patch replicas, resources and image tags. With Helm there is one chart and a values file per environment. ArgoCD applications point at each environment path. Promotion becomes a pull request that changes the image tag or chart version in the next environment's directory, which gives review, approval and history. Application code and configuration often live in separate repositories so that a config change does not require a rebuild and access can be controlled differently. I keep environment differences small and explicit, and use ApplicationSets to generate applications from the directory structure. The aim is a pipeline people trust enough to ship from on a Friday afternoon. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- Why not branch per environment?
- How is promotion done?

A base plus per-environment overlays or values files; promote by changing the image tag or revision per environment.

```
kustomize build overlays/prod
```

```
helm template app ./chart -f values-prod.yaml
```

Interview tip: Directory per environment beats branch per environment.

📚 Official docs

- Kusto Query Language (KQL)
- Helm: Chart best practices
- Helm documentation

certs.eknathalabs.com

Q55 How do you secure ArgoCD itself: RBAC, projects and SSO?

ArgoCD/GitOps

🎙 Spoken answer · ~63 s

ArgoCD has broad power since it can create resources in clusters, so securing it matters. I configure SSO with the corporate identity provider and map groups to roles, and I disable the built-in admin account after bootstrap. AppProjects define what a team may deploy: allowed source repositories, destination clusters and namespaces, and which cluster-scoped resources are permitted. This stops one team from deploying into another team's namespace or creating cluster roles. RBAC policies grant read, sync or override permissions per project. I restrict who can modify projects and who can sync production. Secrets for repository access use least-privilege deploy keys, and the ArgoCD API is not exposed publicly without authentication. I also enable audit logging and keep ArgoCD upgraded, because it is a high-value target. I document the decision so new engineers understand why it is built this way. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- What does an AppProject restrict?
- Why disable the admin account?

Use AppProjects to restrict repos, clusters and namespaces; SSO with group-based RBAC; disable the default admin.

```
argocd proj create team-a --src 'https://git.example/team-a/*' --dest
'https://kubernetes.default.svc,team-a-*'
```

Interview tip: Projects are the multi-tenant boundary.

📚 Official docs

- Kubernetes: RBAC
- OAuth 2.0 / OIDC overview (OpenID)
- Argo CD documentation

certs.eknathalabs.com

Q56 How does Argo Rollouts enable canary and blue-green deployments?

ArgoCD/GitOps

🎙 **Spoken answer · ~55 s**

Argo Rollouts is a controller that extends Kubernetes with a Rollout resource that replaces the Deployment. For a canary, I define steps such as set weight ten percent, pause, analyse, then fifty percent, and so on, and it uses an ingress controller or service mesh to split traffic. For blue-green, it brings up the new version alongside the old, lets me test the preview service, then switches the active service. The key feature is analysis: AnalysisTemplates query metrics from Prometheus or similar, such as error rate and latency, and the rollout automatically promotes if they stay healthy or aborts and rolls back if they breach thresholds. This turns release verification from manual watching into policy. It works well with GitOps, since the Rollout manifest is stored in Git and ArgoCD syncs it.

Likely follow-ups

- What is an AnalysisTemplate?
- When is blue-green better than canary?

A Rollout resource replaces Deployment, shifting traffic by steps and analysing metrics to promote or abort automatically.

```
kubectl argo rollouts get rollout app --watch
```

```
kubectl argo rollouts promote app
```

Interview tip: Analysis makes it automatic.

📚 **Official docs**

- Kubernetes: Deployments
- Google SRE Workbook: Canarying Releases
- Argo Rollouts

certs.eknathalabs.com

Q57 How do you manage multi-cluster deployments and cluster registration in ArgoCD?

ArgoCD/GitOps

🎙 Spoken answer · ~56 s

ArgoCD can deploy to many clusters. In a hub-and-spoke model, one central ArgoCD instance holds credentials for each target cluster, registered as secrets with a service account token. ApplicationSet cluster generators then create an application per cluster, optionally filtered by labels such as environment or region. The advantage is central visibility and a single control plane. The disadvantage is a larger blast radius and a network and credential dependency on the hub. The alternative is an ArgoCD instance in each cluster, which isolates failures and avoids storing remote credentials, but is more to operate. I choose based on scale and security: highly regulated environments often prefer per-cluster instances, while platform teams managing many similar clusters use a hub. In either case, I use projects to limit what each team can deploy to each cluster.

Likely follow-ups

- What is the blast radius of a hub?
- How do labels select clusters?

Register clusters as secrets, use ApplicationSets with cluster generators, and keep a hub-and-spoke or per-cluster instance model.

```
argocd cluster add prod-east --name prod-east
```

```
kubectl get secret -n argocd -l argocd.argoproj.io/secret-type=cluster
```

Interview tip: Hub-and-spoke vs per-cluster instance.

📚 Official docs

- Argo CD: ApplicationSet
- Argo CD documentation

certs.eknathalabs.com

Q58 How do you roll back with ArgoCD and why is git revert preferred?

ArgoCD/GitOps

🎙 Spoken answer · ~56 s

In a GitOps model, Git declares the desired state, so the proper rollback is to revert the offending commit and push it. ArgoCD detects the new commit and syncs the cluster back, leaving a clear audit trail and keeping Git and the cluster consistent. ArgoCD also offers an application rollback to a previous synced revision from its history, which is fast in an emergency, but if auto-sync is enabled it is not allowed or will be overwritten by the next sync, because Git still declares the newer state. So I use the CLI rollback only after disabling auto-sync, and I follow up with a Git revert. For schema changes, rollback also needs backwards-compatible migrations, because reverting the application does not revert the database. Progressive delivery and fast health checks reduce how often rollback is needed.

Likely follow-ups

- Why does UI rollback conflict with auto-sync?
- What about database changes?

🗺 GitOps rollback



Revert the commit in Git so history and cluster agree; UI rollback is temporary when auto-sync is on.

```
git revert <sha> && git push
```

```
argocd app history app
```

```
argocd app rollback app <id>
```

Interview tip: Git is the source of truth.

📚 Official docs

- AWS Builders' Library: Ensuring rollback safety during deployments
- Argo CD documentation

certs.eknathalabs.com

Q59 What is the test pyramid and how does it shape pipeline design?

Testing

🎙 Spoken answer · ~55 s

The test pyramid says a healthy suite has many small, fast unit tests at the base, a moderate number of integration tests that check components working together, and a small number of end-to-end tests through the whole system at the top. Unit tests are quick and precise; end-to-end tests are slow, brittle and hard to debug. In the pipeline I run unit tests on every commit, integration tests with ephemeral dependencies on pull requests, and a small set of critical end-to-end smoke tests after deployment to a test environment. An inverted pyramid, with mostly end-to-end tests, leads to pipelines that take an hour and fail randomly, which erodes trust. I also consider contract tests between services as a faster alternative to full end-to-end tests. The goal is confidence per minute of pipeline time.

Likely follow-ups

- Where do contract tests fit?
- What is the cost of too many e2e tests?

Many fast unit tests, fewer integration tests, very few end-to-end tests; the pipeline runs cheap tests first.

```
pytest -m unit -q
```

```
pytest -m integration
```

Interview tip: Inverted pyramids make slow, flaky pipelines.

📚 Official docs

- Martin Fowler: Continuous Delivery
- Testcontainers / test pyramid (Martin Fowler)

certs.eknathalabs.com

Q60 How do you test infrastructure as code in a pipeline?

Testing

🎙 Spoken answer · ~56 s

I layer the checks. First static: formatting, validation and linting to catch syntax and provider mistakes. Second, security and policy scanning of the code and the generated plan with tools like Checkov or Open Policy Agent, enforcing rules such as no public storage and mandatory encryption and tags. Third, a plan on every pull request, posted as a comment so reviewers see the exact changes. Fourth, for modules, integration tests that create real resources in a sandbox account, assert behaviour and tear everything down, using a framework like Terratest or the native test command. Apply only runs from main after approval, using the saved plan. I also add cost estimation to the review. Not everything can be mocked, so a small number of real deployments in an isolated account gives the most reliable confidence.

Likely follow-ups

- What do you check in a plan?
- How do you clean up test resources?

Validate syntax, lint, policy-check the plan, and run integration tests in a temporary environment.

```
terraform fmt -check && terraform validate
```

```
tflint
```

```
terraform plan -out tfplan && conftest test tfplan.json
```

Interview tip: Plan is the unit test of IaC.

📚 Start here (official docs)

- GitHub Actions documentation

certs.eknathalabs.com

Q61 How do you handle test data and ephemeral environments for integration tests?

Testing

🎙 Spoken answer · ~65 s

Shared test databases create flaky tests because runs interfere and data drifts. I prefer disposable environments created per pipeline run: service containers or Testcontainers for dependencies like databases and queues, or a per-pull-request namespace in Kubernetes that deploys the application with its dependencies and is destroyed when the pull request closes. Test data is created by the tests themselves or seeded from versioned fixtures, and never copied from production unless it has been anonymised. Each test should set up and clean up its own state, with unique identifiers so parallel runs do not collide. For heavy dependencies I use lightweight fakes or contract-verified stubs. I add time-to-live labels and a cleanup job so abandoned environments do not accumulate cost. This makes tests repeatable, parallelisable and independent of each other. I review the setup regularly so it keeps matching how the team really works. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- Why avoid shared test databases?
- How do you stop environment sprawl?

Use containers or per-PR namespaces with seeded, disposable data and automatic teardown.

```
docker compose up -d db
```

```
kubectl create ns pr-123 && helm install app ./chart -n pr-123
```

Interview tip: Never use shared mutable test databases.

📚 Official docs

- Terraform: Sensitive data in state

certs.eknathalabs.com

Q62 How do you quarantine and manage flaky tests without hiding real failures?

Testing

🎙 Spoken answer · ~56 s

A flaky test fails intermittently without a code change and destroys trust in the pipeline. I first measure: track pass and fail history per test so flakes are identified by data. Blind automatic retries mask the problem, so I limit them and report any test that needed a retry. A confirmed flaky test moves to a quarantine suite that still runs and reports but does not block merges, with a ticket, an owner and a deadline, otherwise quarantine becomes a graveyard. Meanwhile I look for root causes: shared state, time dependence, ordering, race conditions, network calls and resource limits, and fix them, for example by using fake clocks and unique test data. I publish the flake rate as a metric, and set a team target. The principle is that a red build must mean something.

Likely follow-ups

- Why are blanket retries risky?
- What causes most flakiness?

Detect by rerun statistics, move to a quarantine suite with an owner and deadline, and track the flake rate.

```
pytest --reruns 0 -p no:randomly
```

```
pytest --count=50 tests/test_x.py
```

Interview tip: Retries hide, quarantine tracks.

📚 Official docs

- Testcontainers / test pyramid (Martin Fowler)
- Google Testing Blog: Flaky tests at Google

certs.eknathalabs.com

Q63 What is mutation testing or coverage gating, and are coverage thresholds useful?

Testing

🎙 Spoken answer · ~65 s

Code coverage measures which lines executed during tests, but it says nothing about whether the tests verify correct behaviour. A test with no assertions can give high coverage. I use a coverage threshold as a floor and a ratchet, so it cannot decrease, and I apply a stricter requirement to changed lines through diff coverage, which is more meaningful for pull requests. To check test quality, mutation testing introduces small bugs, like flipping a condition, and checks whether tests fail. Surviving mutants show weak tests. It is slow, so I run it on critical modules or nightly, not on every commit. I avoid making coverage a team target to be maximised, since it encourages gaming. Better indicators include escaped defects, and whether tests fail when behaviour changes. The aim is a pipeline people trust enough to ship from on a Friday afternoon. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- What is diff coverage?
- Why can coverage be gamed?

Coverage shows what code ran, not whether tests assert behaviour. Mutation testing checks whether tests catch injected bugs.

```
pytest --cov=app --cov-fail-under=80
```

```
mutmut run
```

Interview tip: Use coverage as a floor, not a goal.

📚 Start here (official docs)

- GitHub Actions documentation

certs.eknathalabs.com

Q64 How do you scan for secrets in repositories and block them in CI?

Security

🎙 Spoken answer · ~56 s

I use secret scanning at two points. A pre-commit hook catches secrets on the developer's machine before they enter history, and a CI job scans each pull request and fails if it finds a credential, with a baseline file for known false positives. The platform's push protection can also reject pushes containing known token formats. The most important rule is that removing a secret from the latest commit is not enough: it stays in history and may already have been copied, so the response is to revoke and rotate the credential immediately, then clean history if needed and review access logs for misuse. Prevention matters more than detection, so I steer teams to workload identity and a secrets manager so there are fewer long-lived secrets to leak. I also scan container images and build logs.

Likely follow-ups

- Why rotate rather than delete?
- What reduces the number of secrets?

Use pre-commit and CI secret scanners, block on detection, rotate any leaked secret and scan history.

```
gitleaks detect --source . --redact
```

```
gitleaks protect --staged
```

Interview tip: A leaked secret must be rotated, not just deleted.

📚 Official docs

- gitleaks

certs.eknathalabs.com

Q65 What is an SBOM and how do you generate and use it in a pipeline?

Security

🎙 Spoken answer · ~58 s

An SBOM is an inventory of the components inside a build, including libraries, versions and licences, in a standard format like SPDX or CycloneDX. I generate it during the build with a tool such as Syft, attach it to the image in the registry as an attestation, and store it alongside the release. Its value shows when a new vulnerability is announced: instead of searching every repository, I can query SBOMs to see which deployed artifacts contain the affected component, and who owns them. In the pipeline I can also scan the SBOM against vulnerability databases, enforce licence policies and compare it to the previous release to detect unexpected new dependencies. Some customers and regulators now require SBOMs. The caveat is that an SBOM is only useful if it is accurate and kept up to date for each release.

Likely follow-ups

- Which formats exist?
- How does it help when a new CVE is published?

A software bill of materials lists components and versions in an artifact, enabling vulnerability lookup and licence checks.

```
syft packages app:1.4.2 -o spdx-json > sbom.json
```

```
grype sbom:sbom.json
```

Interview tip: Store it with the artifact.

📚 Official docs

- SPDX specification
- CycloneDX

certs.eknathalabs.com

Q66 How do you apply SAST, DAST and dependency scanning without drowning developers in noise?

Security

🎙 **Spoken answer · ~56 s**

Each tool covers different risks: static analysis examines source for insecure patterns, dependency scanning finds known vulnerable libraries, and dynamic testing probes a running application. To avoid alert fatigue, I start in report-only mode, tune rules to remove noise, and then enforce gradually. I fail the build only on high or critical findings that are new in the pull request, and treat the existing backlog separately with a remediation plan and deadlines. Results appear as comments or annotations in the pull request, where developers already work, with clear fix guidance. Exceptions require a justification, an owner and an expiry date. I also prioritise by exploitability and reachability, not just severity score. Dynamic scans run against a staging deployment on a schedule, because they are slower. Finally I track metrics like mean time to remediate.

Likely follow-ups

- What does fail on new findings mean?
- How do you manage exceptions?

Tune rules, fail only on high severity new findings, show results in pull requests and track exceptions with expiry.

```
semgrep --config auto --error --severity ERROR
```

```
trivy fs --severity HIGH,CRITICAL --exit-code 1 .
```

Interview tip: Fail on new, not old.

📚 **Official docs**

- OWASP ZAP
- GitHub: About code scanning
- Semgrep documentation

Arts.eknathalabs.com

Q67 How do you secure the CI/CD pipeline itself against supply chain attacks?

Security

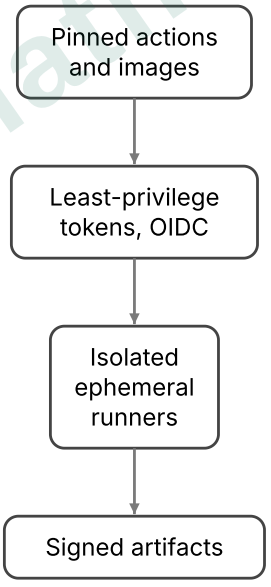
🎙 Spoken answer · ~64 s

The pipeline holds credentials and builds what runs in production, so it is a prime target. I apply several layers. Source: protected branches, required reviews, signed commits for sensitive repositories. Dependencies: lock files, hash verification, a private proxy registry, scanning, and pinned versions for build tools and actions. Execution: ephemeral, isolated runners, short-lived credentials from workload identity, least-privilege tokens per job, and no secrets exposed to untrusted pull request code. Output: reproducible builds where possible, signed artifacts with provenance, and admission control that only accepts artifacts from the official pipeline. I also restrict who can edit pipeline definitions, since changing a pipeline is equivalent to running code with its permissions, and I log and alert on unusual changes. Regular reviews against frameworks like SLSA help prioritise improvements. I document the decision so new engineers understand why it is built this way. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- What does SLSA define?
- Why are pipeline edits sensitive?

🗺 Pipeline hardening



Pin dependencies and actions, least-privilege tokens, isolated ephemeral runners, signed artifacts and protected branches.

```
npm ci --ignore-scripts
```

```
pip install --require-hashes -r requirements.txt
```

certs.eknathalabs.com

Interview tip: Treat the pipeline as production.

📚 Official docs

- GitHub Actions: Security hardening
- SLSA specification

Q68 How do you enforce policy as code for Kubernetes manifests in the pipeline?

Security

🎙 Spoken answer · ~62 s

Policy as code expresses rules, such as no privileged containers, resource limits required, no latest tags and approved registries only, in a machine-checkable language. In the pipeline, I run a tool like Conftest with Open Policy Agent policies, or the Kyverno CLI, against rendered manifests so developers get fast feedback in the pull request. The same policies are enforced in the cluster by an admission controller, because pipeline checks can be bypassed by direct kubectl access. I begin in audit mode to measure violations, fix common ones, then switch to enforce. Policies live in a version-controlled repository with tests, and exceptions are explicit and time-limited. This keeps standards consistent without relying on manual review, and gives auditors clear evidence of controls. I review the setup regularly so it keeps matching how the team really works. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- Why enforce at admission as well?
- How do you roll out policies safely?

Validate manifests with OPA/Conftest or Kyverno CLI in CI and enforce the same rules at admission in the cluster.

```
conftest test deploy/*.yaml
```

```
kyverno apply policies/ --resource deploy/
```

Interview tip: Shift left, enforce at the gate.

📚 Official docs

- Conftest
- Open Policy Agent documentation

certs.eknathalabs.com

Q69 How do you manage the approvals and separation of duties required by compliance in CI/CD?

Security

🎙 Spoken answer · ~64 s

Compliance frameworks usually ask for change control, peer review, separation of duties and evidence. In a modern pipeline I satisfy these through automation rather than paperwork. Branch protection requires pull request approval from someone other than the author and passing checks. Deployments to production run through a controlled pipeline with environment approvals by a defined group, and the pipeline records who approved, what was deployed and the test results, creating an audit trail automatically. Developers do not have standing production access; the pipeline's identity does the deployment. Emergency changes follow a documented break-glass process with after-the-fact review. I store pipeline logs and approvals for the retention period. This approach shows auditors a consistent, enforced process, and it is usually faster than manual change advisory boards. The aim is a pipeline people trust enough to ship from on a Friday afternoon. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- How do you evidence approvals?
- What is break-glass access?

Automate evidence, require peer review and protected branches, and separate who writes from who approves production.

```
# branch protection: require 1 approval, status checks, linear history
```

Interview tip: Pipelines can generate audit evidence.

📚 Start here (official docs)

- GitHub Actions documentation

certs.eknathalabs.com

Q70 How do you implement blue-green deployment in a pipeline and what are the risks?

Pipeline design

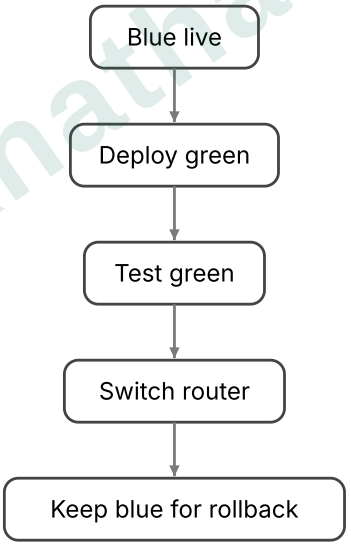
🎙 Spoken answer · ~56 s

Blue-green keeps two identical environments. Blue serves live traffic while I deploy the new version to green, run smoke and integration tests against it, and then switch the router, load balancer or service selector to green. If problems appear, switching back is nearly instant. The benefits are near-zero downtime and fast rollback. The risks are cost, since you run two environments at least briefly, and state. Database changes must be backwards compatible so both versions can work with the same schema, which means using expand and contract migrations. Long-lived connections, sessions and caches also need thought. Also, an all-at-once switch exposes every user to a defect at the same time, so for higher-risk changes I prefer a canary, which limits the blast radius. I keep the blue environment until the new version has proven stable.

Likely follow-ups

- What is expand and contract?
- When is canary better?

🗺 Blue-green



Deploy to the idle environment, test, switch traffic, keep the old one for rollback; database compatibility is the main risk.

```
kubectl apply -f green.yaml
```

```
kubectl patch svc app -p '{"spec":{"selector":{"version":"green"}}}'
```

Interview tip: Schema changes must be backwards compatible.

certs.eknathalabs.com

📚 **Official docs**

- Argo Rollouts
- Martin Fowler: BlueGreenDeployment

Q71 How do you handle configuration and secrets differences across environments without rebuilding?

Pipeline design

🎙 **Spoken answer · ~65 s**

The same immutable artifact should run in every environment, which means environment-specific values cannot be baked into it. I follow the twelve-factor approach: configuration comes from the environment at runtime. Non-sensitive settings are supplied by Helm values, Kustomize overlays or config maps stored in Git per environment. Secrets come from a secret manager and are injected at deploy or runtime, using an external secrets operator, a CSI driver or workload identity, never committed to Git in plain text. I keep the set of configuration keys identical across environments and validate it in the pipeline, so missing settings fail early. Feature flags handle behavioural differences. This ensures that what was tested in staging is exactly what runs in production, apart from explicit configuration, which makes bugs reproducible and rollbacks safe. I document the decision so new engineers understand why it is built this way. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- Why build once?
- How are secrets injected at runtime?

Build once, inject environment configuration at deploy time from config maps, parameter stores and secret managers.

```
helm upgrade app ./chart -f values-prod.yaml
```

```
kubectl create secret generic app --from-literal=...
```

Interview tip: Same artifact, different config.

📚 **Start here (official docs)**

- GitHub Actions documentation

certs.eknathalabs.com

Q72 How do you design pipelines for a microservice that depends on shared libraries?

Pipeline design

🎙 Spoken answer · ~58 s

Shared libraries create coupling, so I treat them like products with semantic versions, changelogs and their own pipelines that test and publish to an internal registry. Services pin specific versions and an update bot opens pull requests when a new version appears, so each service upgrades through its own pipeline with its own tests. For critical libraries I run downstream compatibility checks that build a sample of dependent services against the new version before releasing. Contract tests help when the shared element is an API or schema. I avoid forcing lockstep releases of all services, which defeats independent deployability. Deprecation policy matters: announce, support two versions for a window, then remove. If changes happen very frequently across many services, that may be a sign the code belongs in a monorepo or inside a service instead of a shared library.

Likely follow-ups

- Why avoid lockstep releases?
- How do update bots help?

Version and publish libraries, use dependency updates automatically, and run compatibility or contract tests downstream.

```
npm publish --provenance
```

```
renovate # opens bump PRs
```

Interview tip: Avoid lockstep releases.

📚 Official docs

- Jenkins: Pipeline

certs.eknathalabs.com

Q73 How do you implement automated rollback based on metrics after a deployment?

Pipeline design

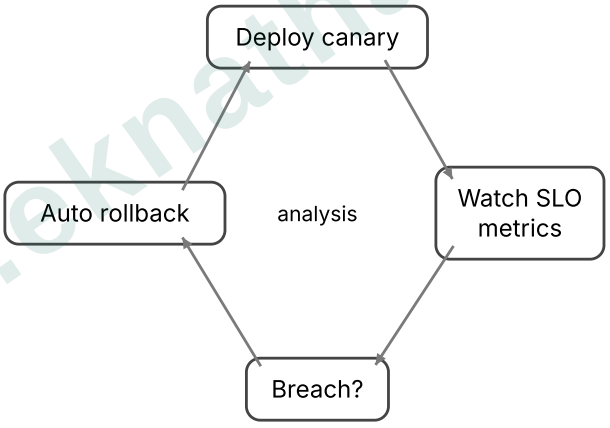
🎙 Spoken answer · ~56 s

Automated rollback needs clear definitions of healthy. I choose a small set of strong signals that reflect user experience, such as error rate, latency percentile and saturation, and compare the new version with the baseline over a bake period of several minutes after traffic shifts. The pipeline or a rollout controller queries the monitoring system and decides: promote if healthy, abort and revert if thresholds are breached. Kubernetes readiness probes and rollout status handle basic failures, while progressive delivery tools handle metric analysis. I avoid hair-trigger thresholds that cause rollbacks on noise by requiring sustained breaches and a minimum request volume. Rollbacks must be tested regularly, and database migrations must be compatible. The system should notify the team with the metrics that triggered it, so the failed release is investigated and not simply retried.

Likely follow-ups

- Which signals would you pick?
- How do you avoid false rollbacks?

🗺 Metric-based rollback



Define SLO-based health checks, observe a bake period, and automatically revert if error rate or latency breach thresholds.

```
kubectl rollout status deploy/app --timeout=300s
```

```
kubectl rollout undo deploy/app
```

Interview tip: Choose a few strong signals.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Deployments
- AWS Builders' Library: Ensuring rollback safety during deployments
- AWS Builders' Library: Automating safe, hands-off deployments

---

Q74 **What is a deployment freeze or change window, and how do you handle it in modern CI/CD?** Pipeline design

🎙 **Spoken answer · ~58 s**

A deployment freeze restricts production changes during risky periods such as major sales events or holidays. In a modern pipeline I implement it as an enforced rule instead of a convention, for example a check that blocks production deployment jobs when a freeze flag or calendar entry is active, with an override requiring approval and a reason. Hotfixes for security or outages go through an emergency path with extra review and logging. I would also challenge the underlying need: teams that deploy small changes frequently with good rollback tend to have less risk per change, so long freezes can actually increase risk by batching changes. During a freeze I keep merging to main and deploying to non-production, so the backlog is not a large release when the freeze ends. Communication of freeze dates is part of the process.

Likely follow-ups

- How is a freeze enforced?
- Why can freezes increase risk?

A period when production changes are restricted. Automate enforcement and keep an emergency path.

```
# pipeline rule: block prod deploys if calendar freeze=true unless label=hotfix
```

Interview tip: Frequent small releases reduce the need for freezes.

📚 **Start here (official docs)**

- GitHub Actions documentation

certs.eknathalabs.com

Q75 How do you build a pipeline for database schema changes with zero downtime?

Pipeline design

🎙 Spoken answer · ~57 s

For zero downtime, the application and schema must work together across versions. I use the expand and contract pattern. In the expand phase I add new columns or tables in a backwards-compatible way, deploy code that writes to both and reads from the new structure, and backfill data in batches. Only after all instances run the new code do I contract by removing old columns in a later release. Migrations are versioned scripts in source control run by a tool such as Flyway or Alembic, executed as a controlled step before or after the rollout with locks and timeouts to prevent long blocking operations. I test migrations against a copy of production-sized data, and take a backup or snapshot. Destructive changes are separated from application releases so rollback of the app does not require rollback of data.

Likely follow-ups

- What is expand and contract?
- How do you avoid long locks?

Use versioned migrations with expand and contract, run them as a controlled step, and keep changes backwards compatible.

```
flyway migrate
```

```
alembic upgrade head
```

Interview tip: Never couple a destructive change to the app release.

📚 Start here (official docs)

- GitHub Actions documentation

certs.eknathalabs.com

Q76 How do you promote releases across environments using the same artifact and gates?

Pipeline design

🎙 Spoken answer · ~55 s

Promotion means moving a release candidate forward without rebuilding it. The pipeline builds once, pushes the image, and records its digest. Deploying to dev is automatic; promotion to staging happens after tests pass; and promotion to production requires gates such as smoke tests, vulnerability policy, change approval and sometimes a soak period. Technically promotion is updating a pointer: the image digest or version referenced by the environment's manifests in Git, or copying the same image to a production registry. Since the artifact is identical, results from staging are meaningful. I keep an audit record of which digest is in each environment and when. I also ensure each stage verifies the deployment with health checks, and that failures stop the promotion automatically. This is faster, safer and more traceable than rebuilding per environment.

Likely follow-ups

- Why not rebuild for production?
- What gates would production require?

🗺 Promotion



Build once, deploy the same digest to dev, staging and production with automated tests and approvals between stages.

```
crane copy reg/app@sha256:... reg/prod/app@sha256:...
```

```
argocd app set app --helm-set image.digest=sha256:...
```

Interview tip: Promotion changes pointers, not binaries.

📚 Start here (official docs)

- GitHub Actions documentation

certs.eknathalabs.com

Q77 A pipeline works on one runner but fails on another. How do you investigate?

Troubleshooting

🎙 **Spoken answer · ~56 s**

A difference between runners usually points to environment drift. I start by comparing the logs of a good and a bad run for tool versions, operating system, environment variables, installed packages, available disk and memory, and timestamps. Then I try to reproduce locally by running the same container image the job uses. Common causes are unpinned tool versions, an older cached dependency, different CPU architecture, missing network access or credentials on one runner pool, file permission or case-sensitivity differences, and leftover state on a long-lived runner. The permanent fix is to make the build hermetic: run jobs in a pinned container image, pin tool and dependency versions, use ephemeral runners and avoid relying on pre-installed software. I also add a step that prints key versions to the log so future differences are easy to spot.

Likely follow-ups

- Why do ephemeral runners help?
- What would you print for diagnostics?

Compare tool versions, environment variables, OS, permissions, network and caches; make the environment reproducible.

```
env | sort
```

```
docker run --rm -it $BUILD_IMAGE bash
```

Interview tip: Reproduce in the same image.

📚 **Start here (official docs)**

- GitHub Actions documentation

certs.eknathalabs.com

Q78 A deployment is stuck in progress and pods never become ready. What do you check?

Troubleshooting

🎙 **Spoken answer · ~59 s**

I begin with rollout status and then describe the pods, because events usually name the cause. Image pull errors show as ErrImagePull or ImagePullBackOff, pointing to a wrong tag, missing registry credentials or network issue. Pending pods suggest insufficient resources, unsatisfied node selectors, taints or an unbound volume. CrashLoopBackOff means the container starts and exits, so I read logs including the previous instance, and check configuration, missing environment variables or secrets, and database connectivity. If pods run but never become ready, the readiness probe is failing, perhaps due to the wrong path or port, a slow start needing a startup probe, or a dependency being down. I also check that the new ReplicaSet respects maxUnavailable and surge settings, and whether a quota or admission policy blocked creation. Once the cause is found I fix it in Git, not by editing live objects.

Likely follow-ups

- What does ImagePullBackOff indicate?
- When do you use a startup probe?

Describe the pods, check events, probes, image pulls, resource requests and config or secret mounts.

```
kubectl rollout status deploy/app
```

```
kubectl describe pod -l app=app
```

```
kubectl logs deploy/app --previous
```

Interview tip: Events first.

📚 **Official docs**

- Kubernetes: Deployments

certs.eknathalabs.com

Q79 Builds suddenly became slow after a dependency update. How do you diagnose?

Troubleshooting

🎙 **Spoken answer · ~65 s**

I start with data: compare step durations before and after the change in the pipeline's timing view. If dependency installation is slow, I check whether the cache key changed so the cache missed, whether a registry or proxy is slow or rate limiting, or whether the new dependency pulls a large transitive tree or compiles native code from source. If the build step is slower, a different compiler version or a new plugin may be the cause. I bisect by reverting the dependency bump or testing commits in between. Fixes can include restoring the cache, using a mirror, pinning versions, adding prebuilt binaries, parallelising or removing unneeded dependencies. Afterwards I add pipeline duration to a dashboard with an alert on regressions, so slowdowns are caught the same day. I review the setup regularly so it keeps matching how the team really works. I would also measure the effect, so the change is justified by data.

Likely follow-ups

- How do you tell a cache miss?
- How do you bisect a slowdown?

Compare timing per step, check cache hits, dependency resolution and network, and bisect the change.

```
time npm ci
```

```
docker build --progress=plain .
```

Interview tip: Timing data per step first.

📚 **Official docs**

- GitHub Actions: Caching dependencies
- Renovate documentation

certs.eknathalabs.com

Q80 A rollback did not fix the problem. What could explain that?

Troubleshooting

🎙 **Spoken answer · ~60 s**

If reverting the application did not resolve the issue, the cause probably lies in state or in something outside the deployed code. Possibilities include a database migration that is not backwards compatible or corrupt data written by the new version, a configuration or secret change applied separately, a feature flag turned on at the same time, a changed upstream dependency or third-party API, an expired certificate or credential, a cache or queue holding bad data, or an infrastructure change like a network rule. I compare the timeline of all changes around the start of the incident, not only deployments, using the change log and audit trails. Then I test hypotheses, for example disabling the flag or clearing the poisoned cache. This is why I try to make changes small, record every change in one timeline, and design migrations that tolerate rolling back the application.

Likely follow-ups

- What other changes do you check?
- How do you make rollbacks safer?

State outside the code changed: database migrations, configuration, feature flags, caches, dependencies or data.

```
kubectl rollout history deploy/app
```

```
git diff v1.4.1 v1.4.2 -- config/
```

Interview tip: Rollback only reverts code.

📚 **Official docs**

- Kubernetes: Deployments
- AWS Builders' Library: Ensuring rollback safety during deployments

certs.eknathalabs.com

Q81 Terraform apply in the pipeline failed halfway. How do you recover safely?

Troubleshooting

🎙 **Spoken answer · ~59 s**

A partial failure leaves some resources created and recorded in state while others are missing. First I make sure nothing is still running and check the lock. If the lock is stale and I have verified the run is dead, I release it. Then I read the error: quota limits, permissions, dependency ordering and API throttling are typical. I review the state to see what exists, and run a plan, which should show only the remaining changes because Terraform tracks what succeeded. If a resource was created in the cloud but not recorded due to a crash, I import it instead of creating a duplicate. After fixing the cause, I apply again, preferably through the same pipeline so there is an audit trail. To reduce risk, I keep changes small, use remote state with locking and versioning, and avoid manual console changes.

Likely follow-ups

- What if a resource exists but is not in state?
- Why keep state versioned?

Do not rerun blindly: check state and lock, review what was created, then re-plan and apply the remainder.

```
terraform state list
```

```
terraform plan
```

```
terraform force-unlock <id>
```

Interview tip: State is the source of truth, so protect it.

📚 **Official docs**

- Terraform CLI: apply

certs.eknathalabs.com

Q82 Docker builds work locally but fail in CI with permission or network errors. What do you check?

Troubleshooting

🎙 **Spoken answer · ~58 s**

Differences between my laptop and CI are usually about identity, network or context. For permission errors I check which user runs the build, whether the Docker socket or daemon is available, file ownership in the workspace and any read-only mounts. For network errors I check DNS, proxy settings, firewall or egress rules on the runner, and whether private registries and package indexes are reachable. Registry issues often come from missing or expired credentials, or from anonymous pull rate limits on public base images, which I solve by authenticating or mirroring base images in our own registry. I also compare the build context: a missing file due to dockerignore or a case-sensitive filesystem can break a build. Running the build with plain progress output and no cache reveals the failing step, and I reproduce it by running the same image locally.

Likely follow-ups

- What causes anonymous pull failures?
- How do you mirror base images?

Check runner user and permissions, registry credentials, proxy and DNS, build context size and rate limits.

```
docker info
```

```
docker login reg.example.com
```

```
docker build --no-cache --progress=plain .
```

Interview tip: Look at auth and rate limits first.

📚 **Official docs**

- Docker documentation

certs.eknathalabs.com

Q83 ArgoCD shows an application as OutOfSync forever even after syncing. What is happening?

Troubleshooting

🎙 **Spoken answer · ~59 s**

If an application is perpetually OutOfSync, something keeps changing live fields so they differ from what Git declares. I run the app diff to see exactly which fields differ. Typical causes: a horizontal pod autoscaler or other controller changing replica counts, a mutating admission webhook or service mesh injecting sidecars and annotations, default values added by the API server that my manifest omits, a Helm chart rendering random values such as generated secrets or timestamps on each render, or a custom resource whose controller writes into the spec. Fixes depend on the cause: remove the field from the manifest if the controller owns it, configure ignore differences for that path, enable server-side apply, or make the template deterministic. I avoid ignoring broad areas because that can hide real drift. Cleaning noise is important so a real out-of-sync status keeps its meaning.

Likely follow-ups

- How do you read an app diff?
- What is server-side apply?

Usually a controller or webhook mutates fields, or defaults differ from the manifest. Configure ignore differences or fix the manifest.

```
argocd app diff app
```

```
argocd app get app --show-operation
```

Interview tip: Read the diff.

📚 **Official docs**

- Argo CD documentation

certs.eknathalabs.com

Q84 Intermittent pipeline failures show 'connection reset' or timeouts when downloading dependencies. What do you do?

Troubleshooting

🎙 **Spoken answer · ~58 s**

Intermittent download failures usually come from the public registry being slow or rate limited, or from the runner's network or NAT having limits. I first check whether failures correlate with time, runner pool or specific hosts. The durable fix is to reduce reliance on the public internet: run an internal caching proxy or artifact repository that mirrors package registries and container base images, so builds fetch from a nearby, reliable source and still work when upstream is down. I also enable dependency caching keyed by lock files, configure sensible retries with exponential backoff, and set timeouts, instead of retrying whole pipelines manually. On the network side I check NAT gateway port exhaustion, DNS resolver performance and egress firewall rules. Monitoring pipeline failure reasons by category helps me see whether infrastructure flakiness is a trend and justify the fix.

Likely follow-ups

- Why use an internal mirror?
- What is NAT port exhaustion?

Add a caching proxy or mirror, retries with backoff, pin versions and check runner network egress.

```
npm config set fetch-retries 5
```

```
pip install --retries 5 -r requirements.txt
```

Interview tip: Reduce dependence on the public internet.

📚 **Official docs**

- RFC 9293: TCP
- AWS Builders' Library: Timeouts, retries and backoff with jitter

certs.eknathalabs.com

Q85 A secret rotation broke deployments across many pipelines. How do you prevent a repeat?

Troubleshooting

🎙 Spoken answer · ~55 s

The root issue is usually that the secret was copied into many places and each pipeline had its own stale value. I would centralise the secret in a manager and have pipelines reference it by name, so a rotation updates one source of truth. Rotation should support an overlap period where both old and new credentials are valid, so consumers can pick up the new one without a hard cutover. Better still, remove static secrets where possible by using short-lived credentials from workload identity. I inventory where each secret is used, announce rotations, and rehearse them in staging. After the incident, I add monitoring for authentication failures, an automated test that validates the credentials, and a runbook for rotation and rollback. This turns rotation from a risky event into a routine, low-impact operation.

Likely follow-ups

- Why overlap old and new credentials?
- How do you find all users of a secret?

Centralise secrets in a manager, reference them by name, rotate with overlap, and test rotation in staging.

```
aws secretsmanager rotate-secret --secret-id ci/registry
```

Interview tip: Dual validity during rotation.

📚 Official docs

- Vault: Dynamic database secrets
- OWASP: Secrets Management Cheat Sheet

certs.eknathalabs.com

Q86 Design the approach to deploy 50 microservices with consistent pipelines but team autonomy.

Pipeline design

🎙 Spoken answer · ~55 s

I would build a paved road: a versioned, reusable pipeline template maintained by a platform team that covers building, testing, scanning, signing, publishing and deploying, with sensible defaults. Teams call it with a few inputs, like language and service name, and can extend it with extra steps for their needs. Guardrails that matter, such as security scanning, artifact signing, and deployment gates for production, are enforced centrally through required workflows or admission policies, so they cannot be skipped. The platform versions the template, announces changes and supports an upgrade path with automated pull requests. Service teams own their services and their application-level tests. I would measure adoption and pipeline health, gather feedback regularly, and treat the platform as a product. This gives consistency, speed of onboarding and compliance without removing team ownership.

Likely follow-ups

- What guardrails would be mandatory?
- How do you roll out template changes?

Provide a golden-path template with reusable pipeline components, let teams extend it, and enforce guardrails centrally.

```
uses: org/platform/.github/workflows/build-deploy.yml@v3
```

Interview tip: Paved road, not forced road.

📚 Start here (official docs)

- GitHub Actions documentation

certs.eknathalabs.com